

Laporan Tugas Besar

Strategi Algoritma (IF2211)



Disusun Oleh

Kelompok when yh libur:

Mirza Tsabita Wafa'ana (13524114)

Safira Berlianti (13524128)

Varistha Devi (13524135)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

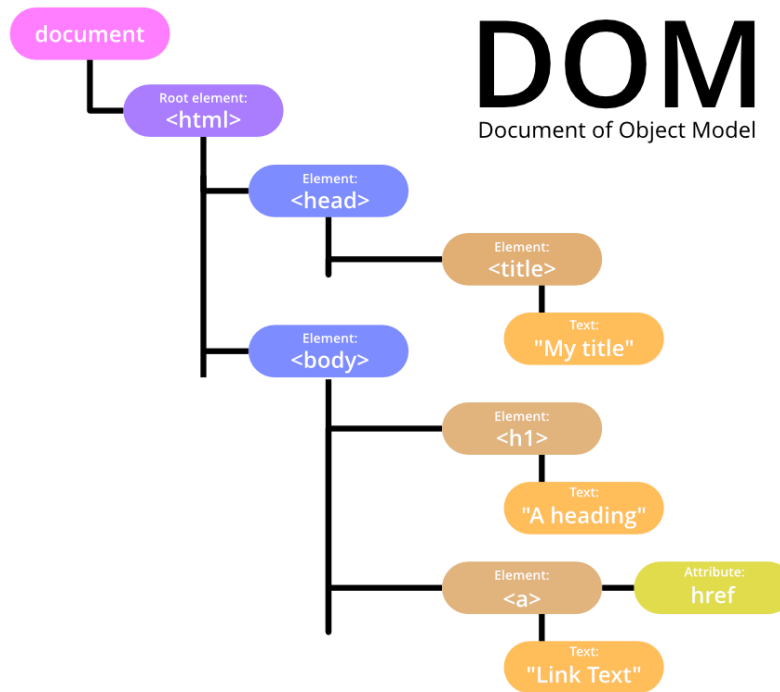
BANDUNG 2026

Daftar Isi

BAB I Deskripsi Tugas.....	4
BAB II Landasan Teori.....	8
2.1 Document Object Model (DOM).....	8
2.2 HTML Parsing.....	8
2.3 CSS Selector & Combinator.....	8
2.4 Algoritma Breadth First Search (BFS).....	9
2.6 Lowest Common Ancestor (LCA).....	10
2.7 Binary Lifting.....	10
BAB III Aplikasi BFS dan DFS.....	11
3.1. Gambaran Umum Persoalan.....	11
3.2. Web Scraping dan Pemodelan Struktur Data Node.....	11
3.3. Implementasi Matching CSS Selector.....	12
3.4. Aplikasi BFS pada Persoalan.....	12
3.5. Aplikasi DFS pada Persoalan.....	13
3.6. Pencatatan Traversal Log Untuk Penjelajahan Struktur DOM.....	13
3.7. Analisis Efisiensi dan Perbandingan BFS vs DFS.....	14
3.8. LCA Binary Lifting.....	14
BAB IV Implementasi dan Pengujian.....	16
4.1. Implementasi.....	16
4.1.1. Backend.....	16
4.1.1.1. main.go.....	16
4.1.1.2. node.go.....	25
4.1.1.3. htmlscraper.go.....	27
4.1.1.4. htmlparser.go.....	29
4.1.1.5. selector.go.....	33
4.1.1.6. parser.go.....	34
4.1.1.7. matcher.go.....	38
4.1.1.8. traverser.go.....	42
4.1.1.9. logger.go.....	47
4.1.1.10. lca.go (bonus).....	49
4.1.2. Interface Program.....	54
4.2. Pengujian.....	56
4.2.1. Testcase 1.....	56
4.2.2. Testcase 2.....	59
4.2.3. Testcase 3.....	63
4.2.4. Testcase 4.....	65
4.2.5. Testcase 5.....	66
BAB V Kesimpulan dan Saran.....	67
5.1. Kesimpulan.....	67

5.2. Saran.....	67
LAMPIRAN.....	68
DAFTAR PUSTAKA.....	70

BAB I Deskripsi Tugas



Gambar 1. Struktur Pohon DOM

<https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/>

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

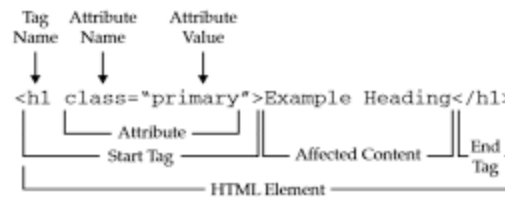
Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. Tags

A graphical overview of the HTML markup syntax shown so far is presented here:



Gambar 2. Struktur Syntax HTML

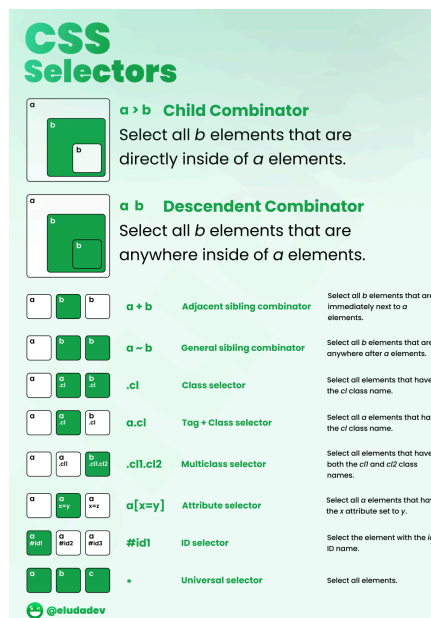
<https://tutorial.techartum.com/htmlTags.html>

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (<>). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang self-closing.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. CSS Selector



Gambar 3. Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak.

Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

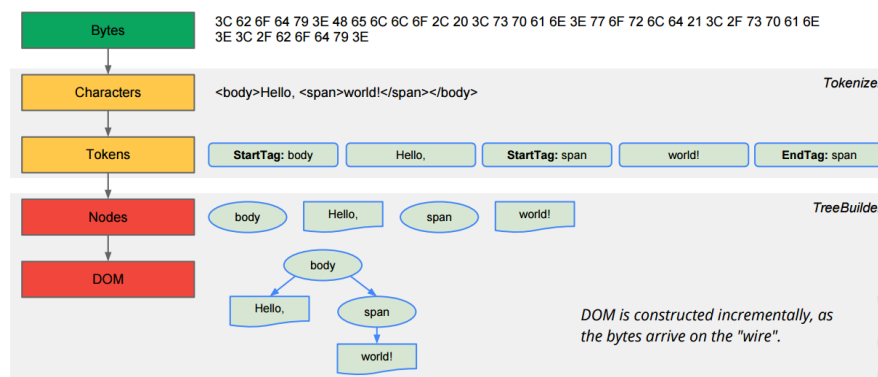
- Tag Selector
Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen `<p>`.
- Class Selector
Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.). Contoh: `.box`
- ID Selector
Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#). Contoh: `#header`
- Universal Selector
Memilih semua elemen HTML. Contoh: `*`

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (`>`)
- Descendent Combinator (`" "`)
- Adjacent Sibling Combinator (`+`)
- General Sibling Combinator (`~`)

3. HTML Parsing

The HTML(5) parser at work...



Gambar 4. Proses Parsing Pohon HTML

<https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen

<html>. Dari node root tersebut, elemen-elemen lain seperti <head> dan <body> akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

Spesifikasi Wajib

Pada tugas ini, mahasiswa diminta untuk membuat aplikasi traversal pohon HTML (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM.

- Buatlah sebuah aplikasi berbasis web yang terdiri dari frontend bebas dan backend dalam bahasa **Go, C#, Rust, atau Typescript** yang mengimplementasikan **algoritma BFS dan DFS** untuk melakukan penelusuran CSS pada pohon DOM.
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna
- Secara umum, berikut adalah input yang diharapkan
 - URL website atau memasuki teks HTML sendiri
 - Pilihan algoritma traversal (BFS / DFS)
 - CSS selector
 - Jumlah hasil yang ingin ditampilkan:
 - Top n kemunculan
 - Semua kemunculan
- HTML yang diperoleh kemudian diparsing menjadi struktur data pohon (DOM Tree).
- Output yang diharapkan adalah sebagai berikut:
 - Aplikasi dapat menampilkan visualisasi struktur pohon DOM, disertai dengan keterangan kedalaman maksimum tree
 - Aplikasi dapat menampilkan highlight pada jalur yang di-traversal oleh algoritma penelusuran yang dipilih untuk menandakan elemen-elemen yang terpengaruh.
 - Aplikasi dapat menampilkan waktu pencarian serta banyak node yang dikunjungi.
 - Setelah melakukan penelusuran, aplikasi menyimpan sebuah *traversal log* yang memiliki informasi tahapan penelusuran yang dilakukan.

BAB II Landasan Teori

2.1 Document Object Model (DOM)

Document Object Model (DOM) adalah representasi terstruktur dari dokumen HTML dalam bentuk pohon (*tree*) yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML sebagai berikut:

- Parent: Elemen yang membungkus elemen lain di dalamnya.
- Child: Elemen yang berada langsung di dalam elemen lain.
- Sibling: Elemen-elemen yang berada pada tingkat hierarki yang sama dan berbagi *parent* yang sama.

Secara umum, file HTML memiliki elemen *root* `<html>` yang bercabang menjadi dua bagian utama, yaitu `<head>` yang berisi metadata dan `<body>` yang berisi konten utama.

2.2 HTML Parsing

HTML Parsing adalah proses membaca dokumen HTML secara berurutan dan mengubahnya menjadi struktur data pohon (DOM Tree) yang merepresentasikan hubungan antar elemen. Proses ini melibatkan pengenalan tag pembuka dan tag penutup untuk menyusun hierarki. Tag yang berada di dalam tag lain akan menjadi *child node*, sedangkan tag pembungkusnya menjadi *parent node*. Hasil akhir dari proses ini adalah sebuah DOM Tree yang memungkinkan elemen-elemen ditelusuri menggunakan algoritma seperti BFS atau DFS.

2.3 CSS Selector & Combinator

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan *style*. Beberapa selector dasar yang umum digunakan meliputi:

- Tag Selector: Memilih elemen berdasarkan nama tag HTML (contoh: `p` memilih semua elemen `<p>`).
- Class Selector: Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.) (contoh: `.box`).
- ID Selector: Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#) (contoh: `#header`).
- Universal Selector: Memilih semua elemen HTML yang ada pada dokumen, ditulis dengan tanda asterisk (*).

Selain itu, juga terdapat attribute selector yang memungkinkan pemilihan elemen berdasarkan keberadaan atau nilai dari atribut tertentu. Berikut adalah operator-operator yang umum digunakan:

- `[attribute]`: Memilih elemen yang memiliki atribut tersebut, tanpa mempedulikan nilainya (contoh: `[target]`).

- `[attribute="value"]`: Memilih elemen dengan nilai atribut yang tepat sama (contoh: `[type="text"]`).
- `[attribute~="value"]`: Memilih elemen yang nilai atributnya mengandung kata tertentu dalam daftar yang dipisahkan spasi (contoh: `[title~="flower"]`).
- `[attribute|="value"]`: Memilih elemen yang nilai atributnya tepat sama atau dimulai dengan kata tersebut diikuti tanda hubung (-) (contoh: `[lang|="en"]`).
- `[attribute^="value"]`: Memilih elemen yang nilai atributnya diawali dengan teks tertentu (contoh: `[href^="https"]`).
- `[attribute$="value"]`: Memilih elemen yang nilai atributnya diakhiri dengan teks tertentu (contoh: `[src$=".png"]`).
- `[attribute*="value"]`: Memilih elemen yang nilai atributnya mengandung substring tertentu di mana saja (contoh: `[class*="content"]`).

Combinator merupakan cara untuk menggabungkan beberapa jenis selector guna menentukan hubungan hierarki antar elemen yang akan dipilih:

- Child Combinator (`>`): Memilih elemen yang merupakan *child* langsung dari *parent* tertentu.
- Descendant Combinator (): Memilih semua elemen keturunan (*descendant*) dari sebuah *parent*, termasuk anak, cucu, dan seterusnya.
- Adjacent Sibling Combinator (`+`): Memilih elemen yang berada tepat setelah elemen lain pada tingkat hierarki yang sama (*sibling* langsung).
- General Sibling Combinator (`~`): Memilih semua elemen *sibling* yang berada setelah elemen tertentu dalam urutan dokumen.

2.4 Algoritma Breadth First Search (BFS)

Breadth First Search (BFS) adalah algoritma penelusuran graf yang menjelajahi struktur pohon secara melebar. Penelusuran dimulai dari *root* dan mengunjungi semua node pada tingkat kedalaman saat ini sebelum berpindah ke tingkat berikutnya, dengan memanfaatkan struktur data queue yang bekerja secara FIFO (First In, First Out). Dalam tugas ini, BFS digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan *selector* yang diberikan. BFS juga diterapkan untuk menelusuri struktur DOM secara melebar, dimulai dari root kemudian mengunjungi node-node pada level yang sama sebelum berpindah ke level berikutnya. Dengan karakteristik tersebut, BFS dapat menemukan elemen yang berada pada kedalaman rendah lebih awal dibandingkan elemen yang berada lebih dalam pada struktur pohon.

2.5 Algoritma Depth First Search (DFS)

Depth First Search (DFS) adalah algoritma penelusuran graf yang menjelajahi struktur pohon secara mendalam. Penelusuran dimulai dari *root* dan masuk sedalam mungkin ke setiap cabang sebelum melakukan backtracking, dengan memanfaatkan struktur data stack yang bekerja secara LIFO (Last In, First Out). Dalam tugas ini, DFS dimanfaatkan untuk menelusuri hierarki DOM dan mencari elemen yang sesuai dengan kriteria CSS selector. DFS juga diterapkan untuk menelusuri struktur DOM secara mendalam, dimulai dari root kemudian mengeksplorasi suatu cabang hingga node terdalam sebelum kembali ke cabang berikutnya. Dengan karakteristik tersebut, DFS sesuai digunakan untuk menjangkau elemen-elemen yang berada di dalam struktur HTML bersarang.

2.6 Lowest Common Ancestor (LCA)

Lowest Common Ancestor (LCA) pada sebuah pohon adalah node terdalam yang menjadi leluhur bersama dari dua node tertentu. Dalam konteks DOM, konsep ini digunakan untuk menentukan elemen HTML terdekat yang membungkus dua elemen spesifik sekaligus. Hal ini berguna dalam analisis struktur dokumen, misalnya untuk mengetahui elemen pembungkus terkecil yang mencakup dua bagian konten yang berbeda.

2.7 Binary Lifting

Binary Lifting adalah teknik untuk mencari leluhur suatu node dalam pohon secara efisien dengan memanfaatkan konsep *dynamic programming* dan lompatan sebesar pangkat dua. Pada teknik ini, untuk setiap node disimpan informasi leluhur ke- 2^k untuk berbagai nilai k , sehingga pencarian dapat dilakukan dengan melompat-lompat sebesar pangkat dua alih-alih naik satu per satu. Dengan cara ini, pencarian LCA dapat dilakukan dalam kompleksitas waktu $O(\log N)$ setelah pra-pemrosesan $O(N \log N)$, di mana N adalah jumlah node dalam pohon.

BAB III Aplikasi BFS dan DFS

3.1. Gambaran Umum Persoalan

Persoalan utama dari tugas besar kali ini yang perlu diselesaikan adalah bagaimana caranya menemukan elemen-elemen HTML tertentu pada suatu halaman web berdasarkan CSS selector, seperti tag, id, attribute selector, ataupun combinator. HTML yang diterima sistem, baik secara mentah ataupun melalui URL yang diinput user, akan diparsing menjadi sebuah DOM tree di mana tiap tag dalam HTML akan direpresentasikan sebagai node yang memiliki hubungan parent-child dengan node lainnya sesuai struktur dokumen.

Sebagai contoh gambaran, HTML berikut

```
<body>
  <div id="container">
    <p class="text">Halo</p>
  </div>
</body>
```

Akan direpresentasikan seperti berikut dalam DOM tree

```
html
├── body
│   └── div#container
│       └── p.text
```

Setelah DOM tree terbentuk, sistem akan melakukan traversal menggunakan DFS atau BFS untuk mencari node elemen-elemen HTML yang memenuhi selector dari user. Kedua algoritma ini diimplementasikan dalam modul traverser.go dan dapat dipilih oleh user melalui interface website. Sebelum penelusuran dimulai, kedalaman maksimum dari seluruh pohon DOM dihitung terlebih dahulu menggunakan fungsi `maxFullTreeDepth()` agar informasi tersebut dapat disertakan dalam hasil penelusuran.

3.2. Web Scraping dan Pemodelan Struktur Data Node

Proses pengolahan data pada aplikasi ini dimulai saat sistem menerima input dari pengguna, yang bisa berupa URL ataupun raw HTML. Jika input yang diberikan adalah URL, maka modul `htmlscraper.go` akan mengirimkan permintaan GET menggunakan protokol HTTP melalui library `net/http`. Setelah koneksi berhasil dan seluruh isi response diterima, data raw HTML tersebut kemudian diteruskan ke modul `htmlparser.go`.

Di dalam `htmlparser.go`, sistem memanfaatkan `html.Tokenizer` untuk memecah dokumen HTML menjadi token-token yang lebih kecil. Untuk menyusun struktur yang merepresentasikan hierarki HTML secara akurat, digunakan struktur data stack. Setiap kali ditemukan tag pembuka, sistem akan membuat objek Node baru, lalu menghubungkannya sebagai anak dari node yang berada di puncak stack. Relasi ini direpresentasikan melalui pointer `Parent` dan `Children` yang didefinisikan dalam `node.go`. Dengan

pendekatan ini, terbentuk struktur hierarkis yang memungkinkan penelusuran baik ke atas (ke parent) maupun ke bawah (ke children).

Setiap objek Node menyimpan informasi seperti nama tag, atribut ID sebagai identitas unik (jika ada), daftar Classes, serta atribut lain yang disimpan dalam bentuk map. Selama proses pembentukan struktur ini, sistem juga menghitung kedalaman setiap node menggunakan fungsi Depth(), sehingga setiap elemen mengetahui posisinya relatif terhadap root.

Setelah seluruh dokumen selesai diproses oleh html.Tokenizer, data yang awalnya berupa teks linear akan berubah menjadi struktur Document Object Model (DOM) Tree yang utuh. Sebelum digunakan lebih lanjut, sistem melakukan validasi melalui fungsi ValidateHTML untuk memastikan bahwa hubungan parent-child dalam pohon sudah konsisten. Jika struktur telah valid, maka DOM Tree siap digunakan oleh algoritma penelusuran seperti BFS atau DFS untuk mencari elemen sesuai dengan kriteria selector yang diberikan oleh pengguna.

3.3. Implementasi Matching CSS Selector

Setelah struktur DOM Tree terbentuk, tahap berikutnya adalah proses pencarian elemen berdasarkan selector yang diberikan pengguna. Proses ini diawali di modul parser.go, di mana string selector tidak langsung digunakan, melainkan diuraikan terlebih dahulu menjadi token-token. Hasil parsing tersebut kemudian diubah menjadi objek SelectorNode yang didefinisikan dalam selector.go. Objek ini tidak hanya menyimpan informasi seperti tag, ID, dan kelas, tetapi juga merepresentasikan hubungan antar bagian selector melalui pointer Prev. Dengan cara ini, selector yang lebih kompleks seperti `div > p` dapat direpresentasikan sebagai rangkaian node yang saling terhubung, sehingga node p tetap mengetahui relasinya dengan div sebagai elemen parent.

Proses pencocokan dilakukan saat algoritma penelusuran (BFS atau DFS) mengunjungi setiap node pada DOM Tree. Pada tahap ini, modul matcher.go berperan dalam menentukan apakah suatu node sesuai dengan selector. Pencocokan dilakukan menggunakan fungsi matchSelectorRecursive. Jika node tersebut memenuhi kriteria, sistem akan melanjutkan pengecekan ke node lain yang berelasi, mengikuti pointer Prev. Dengan demikian, sistem dapat memverifikasi apakah hubungan antar elemen juga sesuai dengan aturan selector (misalnya parent-child atau lainnya). Proses ini berlanjut hingga seluruh rangkaian selector selesai diperiksa (ketika Prev bernilai nil). Jika semua kriteria terpenuhi, maka node tersebut dianggap cocok. Hasil pencarian kemudian disimpan dalam MatchedNodes.

3.4. Aplikasi BFS pada Persoalan

Pengaplikasian BFS untuk persoalan ini menggunakan queue dengan prinsip FIFO. Penelusuran dimulai dari root node, lalu mengunjungi seluruh node pada kedalaman yang sama sebelum berpindah menuju kedalaman selanjutnya. Berikut adalah tahapan dari algoritma BFS yang diimplementasikan:

- 1) Node root dimasukkan ke dalam queue sebagai titik awal penelusuran
- 2) Selama queue tidak kosong, node paling depan diambil dan ditandai sebagai visited
- 3) Node tersebut dicocokkan dengan CSS selector menggunakan fungsi matchF
- 4) Jika cocok, node ditambahkan ke dalam MatchedNodes. Jika jumlah node yang cocok sudah mencapai batas topN, penelusuran dihentikan lebih awal

- 5) Seluruh child dari node yang sedang diproses dimasukkan ke queue (tanpa dibalik urutannya), sehingga penelusuran tetap berjalan dari kiri ke kanan pada setiap level
- 6) Setiap langkah dicatat ke dalam TraversalLog beserta informasi tag, ID, class, kedalaman, dan status kecocokan node

Dikarenakan algoritma ini menelusuri pohon secara melebar terlebih dahulu, BFS cenderung menemukan elemen yang berada lebih dekat dengan root terlebih dahulu. Hal ini bermanfaat ketika user mencari elemen-elemen yang berada pada level hierarki yang lebih tinggi dalam struktur DOM, seperti elemen header.

3.5. Aplikasi DFS pada Persoalan

Pengaplikasian DFS pada persoalan ini menggunakan struktur data stack dengan prinsip LIFO. Penelusuran akan dimulai dari root node, lalu masuk sedalam mungkin ke tiap cabang sebelum melakukan backtracking dan melakukan hal yang sama untuk tiap node yang belum dikunjungi. Berikut adalah tahapan algoritma DFS yang diimplementasikan:

- 1) Node root dimasukkan ke dalam stack sebagai titik awal penelusuran
- 2) Selama stack tidak kosong, node paling atas diambil dan ditandai sebagai visited
- 3) Node tersebut dicocokkan dengan CSS selector menggunakan fungsi matchF
- 4) Jika cocok, node ditambahkan ke dalam MatchedNodes. Jika jumlah node yang cocok sudah mencapai batas topN, penelusuran dihentikan lebih awal
- 5) Seluruh child dari node yang sedang diproses dimasukkan ke stack secara terbalik (dari indeks terakhir ke indeks pertama), sehingga child paling kiri diproses terlebih dahulu sesuai urutan dokumen HTML
- 6) Setiap langkah dicatat ke dalam TraversalLog dengan informasi yang sama seperti pada BFS

Dikarenakan algoritma ini menelusuri pohon secara mendalam sebelum mengunjungi node lainnya di level yang sama, DFS cenderung lebih cepat menemukan elemen yang bersarang atau jauh berada di dalam div bertingkat-tingkat. Penelusuran DFS ini juga mengikuti urutan yang lebih natural terhadap urutan kemunculan elemen dalam dokumen HTML. Dalam tugas besar ini, DFS juga diterapkan untuk menemukan LCA antara dua node dalam sebuah DOM tree.

3.6. Pencatatan Traversal Log Untuk Penjelajahan Struktur DOM

Tiap langkah dari kedua algoritma tersebut akan dicatat secara detail ke dalam struktur data StepLog, dengan informasi yang dicatat meliputi nomor langkah, tag HTML node, ID node, class node, kedalaman node dalam pohon, apakah node cocok dengan selector, nama algoritma yang digunakan, serta indeks node dalam pohon. Log ini kemudian dapat ditampilkan pada output melalui fungsi PrintTraversalLogResult() pada logger.go, yang menyertakan header, detail setiap langkah, dan ringkasan akhir berupa total node yang dikunjungi, total node yang cocok, waktu penelusuran, serta kedalaman maksimum yang dicapai.

3.7. Analisis Efisiensi dan Perbandingan BFS vs DFS

Perbedaan utama keduanya terletak pada cara penelusuran dan penggunaan memori. BFS bekerja secara level-order (horizontal) dengan bantuan queue (FIFO), sehingga semua node pada satu level akan diproses terlebih dahulu sebelum turun ke level berikutnya. Akibatnya, konsumsi memori BFS sangat bergantung pada lebar maksimum pohon. Pada dokumen HTML yang memiliki banyak elemen dalam satu level (misalnya list panjang atau grid), queue bisa menjadi sangat besar.

Sebaliknya, DFS bekerja secara depth-first menggunakan stack (LIFO), dengan menelusuri satu cabang hingga paling dalam sebelum berpindah ke cabang lain. Konsumsi memorinya bergantung pada kedalaman pohon. Dalam praktiknya, struktur HTML modern cenderung lebih lebar daripada dalam, sehingga DFS seringkali lebih hemat memori karena hanya menyimpan satu jalur penelusuran dalam satu waktu.

Dari hasil pengujian, terlihat bahwa BFS lebih cepat menemukan elemen yang berada di dekat root, seperti <header> atau <nav>, karena langsung memproses level atas terlebih dahulu. Sementara itu, DFS lebih efektif untuk menemukan elemen yang berada jauh di dalam struktur, seperti elemen dengan selector spesifik di dalam komponen tertentu, karena langsung menyelam ke cabang terdalam tanpa harus memeriksa semua node di level atas.

Selain itu, urutan hasil pencarian juga berbeda. BFS menghasilkan urutan node berdasarkan kedalaman, sehingga elemen yang lebih dekat ke root muncul lebih dulu. DFS menghasilkan urutan yang mengikuti alur dokumen (pre-order traversal), yang cenderung mirip dengan urutan penulisan HTML, sehingga terasa lebih natural saat digunakan untuk debugging atau membaca struktur.

Secara keseluruhan, tidak ada algoritma yang mutlak lebih baik. BFS unggul untuk pencarian elemen dangkal dan visualisasi berbasis level, sedangkan DFS lebih efisien dari sisi memori dan lebih cocok untuk pencarian mendalam.

3.8. LCA Binary Lifting

Selain fitur pencarian elemen, web kami juga menyediakan fitur untuk menganalisis hubungan antar elemen menggunakan algoritma Lowest Common Ancestor (LCA). Tujuannya adalah untuk menemukan parent terdekat yang sama-sama menaungi dua node dalam struktur DOM. Karena struktur HTML bisa sangat dalam dan kompleks, pendekatan yang digunakan bukan penelusuran linear biasa, melainkan teknik optimasi Binary Lifting yang diimplementasikan di lca.go.

Prosesnya diawali dengan tahap preprocessing setelah DOM Tree selesai dibangun. Sistem melakukan traversal DFS untuk mencatat waktu masuk (tin) dan waktu keluar (tout) dari setiap node. Informasi ini digunakan untuk mengecek dengan cepat apakah suatu node merupakan ancestor dari node lain. Selain itu, sistem juga membangun tabel ancestor, di mana setiap node menyimpan referensi ke ancestorsnya pada jarak tertentu dalam bentuk pangkat dua (2^0 , 2^1 , 2^2 , dan seterusnya). Dengan cara ini, sistem bisa “melompat” ke atas tree dalam beberapa langkah besar, tanpa harus naik satu per satu melalui parent.

Ketika pengguna ingin mencari titik pertemuan dua elemen, fungsi $\text{lca}(u, v)$ akan dijalankan. Jika salah satu node sudah menjadi ancestor dari yang lain, maka hasilnya bisa langsung ditentukan. Namun, jika keduanya berada di cabang yang berbeda, sistem akan menaikkan posisi kedua node menggunakan tabel binary lifting hingga mencapai posisi tepat di bawah ancestor bersama mereka. Setelah itu, ancestor terdekat tersebut dapat ditemukan.

BAB IV Implementasi dan Pengujian

4.1. Implementasi

4.1.1. Backend

4.1.1.1. main.go

Nama	Tipe	Deskripsi
TreeNode	Struct	Representasi node pohon DOM untuk serialisasi JSON ke frontend. Berisi tag, id, classes, nodeIndex, attributes, dan children.
TraversalLogEntry	Struct	Representasi satu entri log traversal (hasil pencarian CSS selector) untuk dikirim ke frontend. Berisi step, tag, id, class, atribut, depth, matched, nodeIndex.
SearchRequest	Struct	Struktur permintaan dari frontend untuk pencarian CSS selector. Berisi URL, HTML, algoritma, selector, dan TopN.
SearchResponse	Struct	Struktur respons untuk pencarian CSS selector. Berisi status, pesan, visited count, max depth, elapsed time, matched count, traversal log, dan tree.
BuildRequest	Struct	Struktur permintaan untuk membangun DOM tree (endpoint /api/build). Berisi URL atau HTML mentah.
BuildResponse	Struct	Struktur respons untuk endpoint build. Berisi status dan tree dalam bentuk TreeNode.
assignIndices	Fungsi	Memberikan indeks unik (pre-order) pada setiap node dalam pohon menggunakan DFS. Mengembalikan map dari pointer node ke indeks.

serializeTree	Fungsi	Mengonversi pohon DOM internal (*Node) menjadi struktur TreeNode untuk serialisasi JSON (dengan indeks).
convertStepLog	Fungsi	Mengonversi slice StepLog (dari traversal) menjadi slice TraversalLogEntry yang siap dikirim ke frontend.
handleBuild	Fungsi (handler)	HTTP handler untuk endpoint /api/build. Menerima URL/HTML, memparse, menyimpan state global, dan mengembalikan tree.
updateGlobalState	Prosedur	Menyimpan root, indexMap, nodeByIndex, dan membangun LCATable ke dalam variabel global setelah parsing HTML.
handleLCA	Fungsi (handler)	HTTP handler untuk endpoint /api/lca. Menerima dua nodeIndex, mencari LCA menggunakan binary lifting, mengembalikan informasi node LCA beserta path A dan B.
getPathToAncestor	Fungsi	Membantu LCA: menghasilkan slice indeks node dari node tertentu sampai ancestor (termasuk ancestor) dengan menelusuri parent pointer.
handleSearch	Fungsi (handler)	HTTP handler untuk endpoint /api/search. Memuat HTML, membangun DOM, menjalankan traversal (BFS/DFS) dengan CSS selector, mengembalikan hasil pencarian.
main	Fungsi	Entry point aplikasi. Menentukan frontendDir (dengan fallback), menyajikan file statis, mendaftarkan HTTP handler, dan menjalankan server di port 8080.

```

package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "os"
    "path/filepath"
    "sync"
)

// buat nampung data dom tree
type TreeNode struct {
    Tag        string        `json:"tag"`
    ID         string        `json:"id"`
    Classes    []string      `json:"classes"`
    NodeIndex  int           `json:"nodeIndex"`
    Children   []*TreeNode   `json:"children"`
}

// buat nyatet log tiap langkah
type TraversalLogEntry struct {
    Step        int           `json:"step"`
    NodeTag     string        `json:"nodeTag"`
    NodeId      string        `json:"nodeId"`
    NodeClass   []string      `json:"nodeClass"`
    Depth       int           `json:"depth"`
    Matched     bool          `json:"matched"`
    NodeIndex   int           `json:"nodeIndex"`
}

// SearchRequest dari frontend
type SearchRequest struct {
    URL         string        `json:"url"`
    HTML        string        `json:"html"`
    Algorithm    string        `json:"algorithm"`
    Selector     string        `json:"selector"`
    TopN        int           `json:"topN"`
}

// SearchResponse buat frontend
type SearchResponse struct {
    Success      bool          `json:"success"`
    Message      string        `json:"message,omiteempty"`
    VisitCount   int           `json:"visitCount"`
    MaxDepth     int           `json:"maxDepth"`
    ElapsedTime  string        `json:"elapsedTime"`
    MatchedCount int           `json:"matchedCount"`
    TraversalLog []TraversalLogEntry `json:"traversalLog"`
    Tree         *TreeNode
}

```

```

}

// BuildRequest buat endpoint /api/build
type BuildRequest struct {
    URL    string `json:"url"`
    HTML   string `json:"html"`
}

type BuildResponse struct {
    Success bool      `json:"success"`
    Message string    `json:"message,omitempty"`
    Tree    *TreeNode `json:"tree"`
}

// global state buat lca
var (
    currentRoot *Node
    currentIndexMap map[*Node]int
    currentNodeByIndex map[int]*Node
    currentLCATable *LCATable
    stateMutex sync.Mutex
)

// buat kasih nomer urut ke tiap node
func assignIndices(root *Node) map[*Node]int {
    indexMap := make(map[*Node]int)
    idx := 0
    var traverse func(n *Node)
    traverse = func(n *Node) {
        if n == nil {
            return
        }
        indexMap[n] = idx
        idx++
        for _, child := range n.Children {
            traverse(child)
        }
    }
    traverse(root)
    return indexMap
}

// ngubah Node jadi TreeNode
func serializeTree(node *Node, indexMap map[*Node]int) *TreeNode {
    if node == nil {
        return nil
    }
    treeNode := &TreeNode{
        Tag: node.Tag,
        ID: node.ID,
        Classes: node.Classes,
    }

```

```

        NodeIndex: indexMap[node],
        Children: []*TreeNode{},
    }
    for _, child := range node.Children {
        treeNode.Children = append(treeNode.Children,
serializeTree(child, indexMap))
    }
    return treeNode
}

// ngubah StepLog jadi TraversalLogEntry
func convertStepLog(logs []StepLog) []TraversalLogEntry {
    entries := make([]TraversalLogEntry, len(logs))
    for i, l := range logs {
        entries[i] = TraversalLogEntry{
            Step: l.Step,
            NodeTag: l.N_Tag,
            NodeId: l.N_ID,
            NodeClass: l.N_Classes,
            Depth: l.Depth,
            Matched: l.Is_matched,
            NodeIndex: l.NodeIndex,
        }
    }
    return entries
}

func handleBuild(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    if r.Method != http.MethodPost {
        json.NewEncoder(w).Encode(BuildResponse{Success: false,
Message: "Method not allowed"})
        return
    }

    var req BuildRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        json.NewEncoder(w).Encode(BuildResponse{Success: false,
Message: "Invalid JSON"})
        return
    }
    if req.URL == "" && req.HTML == "" {
        json.NewEncoder(w).Encode(BuildResponse{Success: false,
Message: "URL or HTML required"})
        return
    }

    var input string
    if req.URL != "" {
        input = req.URL
    } else {

```

```

        input = req.HTML
    }

    root, err := LoadHTML(input)
    if err != nil {
        json.NewEncoder(w).Encode(BuildResponse{Success: false,
Message: err.Error()})
        return
    }
    if err := ValidateHTML(root); err != nil {
        json.NewEncoder(w).Encode(BuildResponse{Success: false,
Message: "HTML structure error: " + err.Error()})
        return
    }

    indexMap := assignIndices(root)
    updateGlobalState(root, indexMap) // simpan state untuk LCA
nanti

    tree := serializeTree(root, indexMap)
    json.NewEncoder(w).Encode(BuildResponse{Success: true, Tree:
tree})
}

// dipanggil setelah parsing html dan assign indices
func updateGlobalState(root *Node, indexMap map[*Node]int) {
    stateMutex.Lock()
    defer stateMutex.Unlock()
    currentRoot = root
    currentIndexMap = indexMap
    // mapping balik
    nodeByIndex := make(map[int]*Node)
    for node, idx := range indexMap {
        nodeByIndex[idx] = node
    }
    currentNodeByIndex = nodeByIndex
    // bangun LCATable
    currentLCATable = buildLCATable(root)
}

// nanganin request lca
func handleLCA(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    if r.Method != http.MethodPost {
        json.NewEncoder(w).Encode(map[string]interface{}{"success": false,
"message": "Method not allowed"})
        return
    }
    var req struct {
        NodeIndexA int `json:"nodeIndexA"`
    }
}

```

```

        NodeIndexB int `json:"nodeIndexB"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
json.NewEncoder(w).Encode(map[string]interface{}{"success": false,
"message": "Invalid JSON"})
        return
    }

    stateMutex.Lock()
    defer stateMutex.Unlock()
    if currentLCATable == nil || currentNodeByIndex == nil {
json.NewEncoder(w).Encode(map[string]interface{}{"success": false,
"message": "Belum ada pohon DOM. Lakukan pencarian terlebih
dahulu."})
        return
    }

    nodeA := currentNodeByIndex[req.NodeIndexA]
    nodeB := currentNodeByIndex[req.NodeIndexB]
    if nodeA == nil || nodeB == nil {
json.NewEncoder(w).Encode(map[string]interface{}{"success": false,
"message": "Node tidak valid"})
        return
    }

    lcaNode := currentLCATable.lca(nodeA, nodeB)
    if lcaNode == nil {
json.NewEncoder(w).Encode(map[string]interface{}{"success": false,
"message": "LCA tidak ditemukan"})
        return
    }

    response := map[string]interface{}{
        "success": true,
        "tag": lcaNode.Tag,
        "id": lcaNode.ID,
        "classes": lcaNode.Classes,
        "depth": currentLCATable.depth[lcaNode],
        "nodeIndex": currentIndexMap[lcaNode],
    }
    json.NewEncoder(w).Encode(response)
}

func handleSearch(w http.ResponseWriter, r *http.Request) {
    // set header json
    w.Header().Set("Content-Type", "application/json")

```

```

        if r.Method != http.MethodPost {
            w.WriteHeader(http.StatusMethodNotAllowed)
            json.NewEncoder(w).Encode(SearchResponse{Success: false,
Message: "Method not allowed"})
            return
        }

        var req SearchRequest
        if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
            w.WriteHeader(http.StatusBadRequest)
            json.NewEncoder(w).Encode(SearchResponse{Success: false,
Message: "Invalid JSON"})
            return
        }

        // validasi input, harus ada url atau html
        if req.URL == "" && req.HTML == "" {
            w.WriteHeader(http.StatusBadRequest)
            json.NewEncoder(w).Encode(SearchResponse{Success: false,
Message: "Either url or html must be provided"})
            return
        }

        // load html dari input
        var input string
        if req.URL != "" {
            input = req.URL
        } else {
            input = req.HTML
        }
        root, err := LoadHTML(input)
        if err != nil {
            w.WriteHeader(http.StatusBadRequest)
            json.NewEncoder(w).Encode(SearchResponse{Success: false,
Message: "Failed to load HTML: " + err.Error()})
            return
        }

        // validasi struktur html
        if err := ValidateHTML(root); err != nil {
            w.WriteHeader(http.StatusBadRequest)
            json.NewEncoder(w).Encode(SearchResponse{Success: false,
Message: "HTML structure error: " + err.Error()})
            return
        }

        // bikin index map
        indexMap := assignIndices(root)

        // simpen state lca
        updateGlobalState(root, indexMap)

```

```

// bikin match function
matchFunc := MakeMatchFunc(req.Selector)

// pilih mau pake cara dfs atau bfs
var algoType AlgorithmType
switch req.Algorithm {
case "DFS":
    algoType = AlgoDFS
default:
    algoType = AlgoBFS
}

// lakuin pencarian
result := Search(root, req.Selector, algoType, matchFunc,
req.TopN, indexMap)

// bikin struktur pohon
tree := serializeTree(root, indexMap)

// konversi log
logEntries := convertStepLog(result.TraversalLog)

// hitung matched count
matchedCount := len(result.MatchedNodes)

// format elapsed time
elapsedMs := float64(result.ElapsedTime.Nanoseconds()) / 1e6
elapsedStr := fmt.Sprintf("%.3f ms", elapsedMs)

response := SearchResponse{
    Success:      true,
    VisitCount:   result.VisCount,
    MaxDepth:     result.MaxDepthVisited,
    ElapsedTime:  elapsedStr,
    MatchedCount: matchedCount,
    TraversalLog: logEntries,
    Tree:         tree,
}

json.NewEncoder(w).Encode(response)
}

func main() {
    frontendDir := "../frontend"
    if _, err := os.Stat(frontendDir); os.IsNotExist(err) {
        log.Fatalf("Frontend directory not found at %s",
frontendDir)
    }

    // handler file statis

```

```

    fs := http.FileServer(http.Dir(frontendDir))
    http.Handle("/", http.HandlerFunc(func(w http.ResponseWriter,
r *http.Request) {
        if r.URL.Path == "/" {
            http.ServeFile(w, r, filepath.Join(frontendDir,
"index.html"))
            return
        }
        fs.ServeHTTP(w, r)
    }))

    // api endpoint
    http.HandleFunc("/api/search", handleSearch)
    http.HandleFunc("/api/build", handleBuild)
    http.HandleFunc("/api/lca", handleLCA)

    log.Println("Server started at http://localhost:8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

```

4.1.1.2. node.go

Nama	Tipe	Deskripsi
Node	Struct	Struktur data utama node dalam pohon DOM. Berisi tag, id, classes, attributes, children, parent, dan insideText.
NewNode	Fungsi (konstruktor)	Membuat instance Node baru dengan tag yang diberikan (diubah ke lowercase) dan menginisialisasi field kosong.
HasClasses	Method (Node)	Memeriksa apakah node memiliki kelas tertentu. Mengembalikan true jika ada.
GetAttribute	Method (Node)	Mengembalikan nilai attribute berdasarkan kunci, serta boolean apakah attribute ditemukan.
SiblingsBeforeNow	Method (Node)	Mengembalikan slice node saudara yang berada sebelum node ini (dalam urutan children parent). Digunakan untuk sibling combinator.
ImmediateSiblingsBeforeNow	Method (Node)	Mengembalikan node saudara

		tepat sebelum node ini (langsung sebelumnya), atau nil jika tidak ada.
Depth	Method (Node)	Menghitung kedalaman node (jumlah langkah ke root) dengan menelusuri parent pointer.

```

package main
import "strings"

type Node struct{
    Tag string //buat nyimpen tag HTML seperti div, p, span, ect.
    ID string //buat nyimpen id (id="...")
    Classes []string //buat nyimpen kelas-kelas (class="...")
    Attributes map[string]string //buat atribut-atribut lain
    Children []*Node //karena node anaknya bisa banyak
    Parent *Node
    InsideText string //isi dalam tag
}

//ctor
func NewNode(tag string) *Node{
    return &Node{
        Tag: strings.ToLower(tag),
        ID: "",
        Classes: []string{},
        Attributes: make(map[string]string),
        Children: []*Node{},
        Parent: nil,
        InsideText: "",
    }
}

//buat ngecek apakah node punya kelas-kelas tertentu
func (n *Node) HasClasses(class string) bool{
    for _, c := range n.Classes{
        if c == class{
            return true
        }
    }
    return false
}

//ngembaliin nilai atribut dari map, atau string kosong kalo gaada
func (n *Node) GetAttribute(key string) (string, bool) {
    val, ok := n.Attributes[key]
    return val, ok
}

```

```

//return semua sanak saudara yang datang sebelum node sekarang,
dipake buat adjacent dan general sibling selector
func (n *Node) SiblingsBeforeNow() []*Node{
    if n.Parent == nil{
        return []*Node{}
    }
    result := []*Node{}
    for _, sibling := range n.Parent.Children{
        if sibling == n{
            break
        }
        result = append(result, sibling)
    }
    return result
}

//ngasih tau siapa exactly saudara (singular) yang datang sebelum
node sekarang
func (n *Node) ImmediateSiblingsBeforeNow() *Node{
    siblings := n.SiblingsBeforeNow()
    if len(siblings) == 0{
        return nil
    }
    return siblings[len(siblings)-1]
}

//buat ngitung kedalaman node
func (n *Node) Depth() int{
    depth := 0
    current := n.Parent
    for current != nil{
        depth++
        current = current.Parent
    }
    return depth
}

```

4.1.1.3. htmlscraper.go

Nama	Tipe	Deskripsi
fetchHTML	Fungsi	Mengambil konten HTML dari URL yang diberikan menggunakan HTTP GET. Mengembalikan string HTML atau error.
LoadHTML	Fungsi	Memuat HTML dari input (bisa

		URL atau raw HTML). Jika input berupa URL, akan di-fetch terlebih dahulu. Mengembalikan root node pohon DOM atau error.
--	--	---

```

package main
import(
    "fmt"
    "io"
    "net/http" //ini dipake buat nge-fetch raw html dari url
    "strings"
    "time"
)

func fetchHTML(url string) (string, error){
    if !strings.HasPrefix(url, "http://") &&
    !strings.HasPrefix(url, "https://"){
        return "", fmt.Errorf("url invalid!")
    }

    client := &http.Client{
        Timeout: 15 * time.Second,
    }

    resp,err := client.Get(url)
    if err != nil{
        return "", fmt.Errorf("gagal fetch url: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK{
        return "", fmt.Errorf("unexpected status code: %d",
resp.StatusCode)
    }

    body, err := io.ReadAll(resp.Body)
    if err != nil{
        return "", fmt.Errorf("gagal baca response body: %w",
err)
    }

    return string(body), nil
}

//basically dia yang handling ngeload html dari user input (baik
dalam bentuk url atau mentahan)
func LoadHTML(input string) (*Node, error){

```

```

    input = strings.TrimSpace(input)
    if input == ""{
        return nil, fmt.Errorf("input kosong!")
    }

    var rawHTML string
    if strings.HasPrefix(input, "http://") ||
strings.HasPrefix(input, "https://"){
        //fetch html dari url input
        html, err := fetchHTML(input)
        if err != nil{
            return nil, err
        }
        rawHTML = html
    }else{
        rawHTML = input
    }

    root := parseHTML(rawHTML)
    if root == nil{
        return nil, fmt.Errorf("gagal parsing html :(")
    }
    return root, nil
}

```

4.1.1.4. htmlparser.go

Nama	Tipe	Deskripsi
parseHTML	Fungsi	Mengurai raw HTML menjadi pohon DOM (*Node) menggunakan tokenizer HTML. Mengembalikan node root atau nil jika gagal.
ValidateHTML	Fungsi	Memeriksa konsistensi struktur pohon DOM (parent-child relationship). Mengembalikan error jika ada ketidaksesuaian.
selfClosingTags	Variabel (map)	Map berisi tag HTML yang bersifat self-closing (tidak memiliki children), digunakan untuk menentukan apakah suatu elemen perlu dipush ke stack.

```
package main
```

```

import (
    "fmt"
    "strings"
    "golang.org/x/net/html" //ini dipake buat tokenization using
html.Tokenizer ajah
)

//convert raw html jadi DOM tree. html.Tokenizer bakalan ngescan
html mentahnya dan ngubah jadi stream of tokens. dari situ kita
bisa bangun node tree kita dengan stack :D
func parseHTML(rawHTML string) *Node {
    if strings.TrimSpace(rawHTML) == ""{
        return nil
    }

    tokenizer := html.NewTokenizer(strings.NewReader(rawHTML))
    //virtual root asal buat container sementara dan pijakan awal
dulu aja, dan biar lebih seragam :v
    virRoot := NewNode("#virtualroot")
    //stack buat nyimpen nodes yang lagi diproses, it also helps
tracks ancestry
    stack := []*Node{virRoot}
    for{
        tt := tokenizer.Next()
        switch tt{
            case html.ErrorToken:
                goto done
            case html.StartTagToken:
                name, hasAttribute := tokenizer.TagName()
                tag := strings.ToLower(string(name))
                node := NewNode(tag)
                node.Parent = stack[len(stack)-1]
                stack[len(stack)-1].Children =
append(stack[len(stack)-1].Children, node)
                //parse semua atribut
                for hasAttribute{
                    var key, value []byte
                    key, value, hasAttribute = tokenizer.TagAttr()
                    k := strings.ToLower(string(key))
                    v := string(value)
                    switch k{
                        case "id":
                            node.ID = v
                        case "class":
                            node.Classes = strings.Fields(v)
                        default:
                            node.Attributes[k] = v
                    }
                }
                //hanya push ke stack kalau bukan self-closing tag
(gapunya children soalnya)

```

```

        if !selfClosingTags[tag]{
            stack = append(stack, node)
        }
    case html.SelfClosingTagToken:
        name, hasAttribute := tokenizer.TagName()
        tag := strings.ToLower(string(name))
        node := NewNode(tag)
        node.Parent = stack[len(stack)-1]
        stack[len(stack)-1].Children =
append(stack[len(stack)-1].Children, node)
        //parse semua atribut
        for hasAttribute{
            var key, value []byte
            key, value, hasAttribute = tokenizer.TagAttr()
            k := strings.ToLower(string(key))
            v := string(value)
            switch k{
            case "id":
                node.ID = v
            case "class":
                node.Classes = strings.Fields(v)
            default:
                node.Attributes[k] = v
            }
        }
    case html.EndTagToken:
        name, _ := tokenizer.TagName()
        tag := strings.ToLower(string(name))
        //pop stack sampe ketemu open tag yang matching, buat
handle kasus tag yang ga ditutup atau nested dengan tag yang sama
        for len(stack) > 1{
            top := stack[len(stack)-1]
            stack = stack[:len(stack)-1]
            if top.Tag == tag{
                break
            }
        }
    case html.TextToken:
        //attach text ke current top node sebagai InsideText
        text := strings.TrimSpace(string(tokenizer.Text()))
        if text != "" && len(stack) > 0{
            top := stack[len(stack)-1]
            if top.InsideText == ""{
                top.InsideText = text
            }else{
                top.InsideText += " " + text
            }
        }
    case html.CommentToken:
        //boleh diskip
        continue

```

```

        case html.DoctypeToken:
            continue
    }
}
done:
//kembaliin anak-anak virtual root, kalau punya satu anak aja
kembaliin itu sebagai root
if len(virRoot.Children) == 1{
    root := virRoot.Children[0]
    root.Parent = nil
    return root
}
if len(virRoot.Children) == 0{
    return nil
}
return virRoot
}

//ini buat basic structural checking pada DOM tree, bakalan return
error string kalau ada issues, empty string kalau aman
func ValidateHTML(root *Node) error{
    if root == nil{
        return fmt.Errorf("html structure error, tree kosong (html
kosong atau invalid)")
    }
    //cek parent-child consistency, pastiin setiap anak node punya
parent yang bener
    var check func(n *Node) error
    check = func(n *Node) error{
        for _, child := range n.Children{
            if child.Parent != n{
                return fmt.Errorf("html structure error, node <%s>
punya parent yang salah", child.Tag)
            }
            if err := check(child); err != nil{
                return err
            }
        }
        return nil
    }
    return check(root)
}

//lists of all html tags yang child-free
var selfClosingTags = map[string]bool{
    "area": true,
    "base": true,
    "br": true,
    "col": true,
    "embed": true,
    "hr": true,

```

```

    "img": true,
    "input": true,
    "link": true,
    "meta": true,
    "param": true,
    "source": true,
    "track": true,
    "wbr": true,
}

```

4.1.1.5. selector.go

Nama	Tipe	Deskripsi
AttributeSelector	Struct	Menyimpan satu kondisi attribute selector: nama atribut, operator, nilai, dan flag case-insensitive (i atau I).
SelectorNode	Struct	Representasi satu komponen selector (misal div, .container, #main) beserta daftar attribute selector, relasi terhadap komponen kiri, dan pointer ke komponen sebelumnya (Prev). Prev membentuk linked list ke arah kiri selector.

```

package main

type AttributeSelector struct {
    Name string
    Operator string
    Value string
    CaseInsensitive bool
}

type SelectorNode struct {
    Tag string
    ID string
    Classes []string
    Attributes []AttributeSelector
    Relation string // "child", "descendant", "adjacent_sibling",
"general_sibling"
    Prev *SelectorNode
}

```

4.1.1.6. parser.go

Nama	Tipe	Deskripsi
ParseSelector	Fungsi	Mengubah string CSS selector menjadi linked list SelectorNode yang berorientasi dari kanan ke kiri (paling spesifik menjadi root linked list).
tokenize	Fungsi	Memecah string selector menjadi token-token (tag, combinator, id, class, attribute) dengan menelusuri karakter per karakter.
convertTokensToNodes	Fungsi	Mengubah slice token menjadi linked list SelectorNode (dari kiri ke kanan sementara), kemudian biasanya diproses lebih lanjut untuk membalik arah.
parseAttribute	Fungsi	Mengurai token attribute selector (misal [data-type="personal"]) menjadi struct AttributeSelector (name, operator, value, case-insensitive flag).
mapRelation	Fungsi	Mengubah simbol combinator (spasi, >, +, ~) menjadi string relasi internal (descendant, child, adjacent_sibling, general_sibling).

```
package main

import "strings"

func ParseSelector(input string) *SelectorNode {
    tokens := tokenize(input)
    head := convertTokensToNodes(tokens)
    current := head
    for current.Prev != nil {
        current = current.Prev
    }
    return current
}
```

```

// ubah string input jadi tokens
func tokenize(input string) []string {
    var isAttribute bool
    var token strings.Builder
    var tokens []string

    for i := 0; i < len(input); i++ {
        char := input[i]

        if isAttribute {
            token.WriteByte(char)
            if char == ']' {
                tokens = append(tokens, token.String())
                token.Reset()
                isAttribute = false
            }
            continue
        }

        if char == '[' {
            if token.Len() > 0 {
                tokens = append(tokens, token.String())
                token.Reset()
            }
            isAttribute = true
            token.WriteByte(char)
        } else if char == ' ' {
            if token.Len() > 0 {
                tokens = append(tokens, token.String())
                token.Reset()
            }
            if len(tokens) > 0 && tokens[len(tokens)-1] != " "
{
                tokens = append(tokens, " ")
            }
        } else if char == '.' || char == '#' {
            if token.Len() > 0 {
                tokens = append(tokens, token.String())
                token.Reset()
            }
            token.WriteByte(char)
        } else if char == '>' || char == '+' || char == '~' ||
char == '*' {
            if token.Len() > 0 {
                tokens = append(tokens, token.String())
                token.Reset()
            }
            tokens = append(tokens, string(char))
        } else {
            token.WriteByte(char)
        }
    }
}

```

```

    }

    if token.Len() > 0 {
        tokens = append(tokens, token.String())
    }
    return tokens
}

// ubah tokens jadi SelectorNode
func convertTokensToNodes(tokens []string) *SelectorNode {
    head := &SelectorNode{}
    current := head

    for _, t := range tokens {
        switch {
            case t == " " || t == ">" || t == "+" || t == "~":
                current.Relation = mapRelation(t)
                newNode := &SelectorNode{}
                current.Prev = newNode
                current = newNode

            case t[0] == '.':
                classPart := t[1:]
                classes := strings.Split(classPart, ".")
                for _, c := range classes {
                    if c != "" {
                        current.Classes = append(current.Classes,
c)
                                }
                    }
                case t[0] == '#':
                    current.ID = t[1:]
                case t[0] == '[':
                    attribute := parseAttribute(t)
                    current.Attributes = append(current.Attributes,
attribute)
                default:
                    current.Tag = t
                }
            }
        }
        return head
    }

    // buat parse atribut
    func parseAttribute(t string) AttributeSelector {
        attribute := AttributeSelector{}
        var builder strings.Builder
        var isVal bool

        for i := 1; i < len(t)-1; i++ {
            char := t[i]

```

```

        if isVal {
            if char == '"' {
                attribute.Value = builder.String()
                builder.Reset()
                isVal = false
            } else {
                builder.WriteByte(char)
            }
            continue
        }

        if i+1 < len(t)-1 && strings.ContainsRune("~|^$*",
rune(char)) && t[i+1] == '=' {
            attribute.Name = builder.String()
            builder.Reset()
            attribute.Operator = string(char) + "="
            i++
        } else if char == '=' {
            attribute.Name = builder.String()
            builder.Reset()
            attribute.Operator = "="
        } else if char == '"' {
            isVal = true
        } else if char == 'i' || char == 'I' {
            attribute.CaseInsensitive = true
        } else if char == ' ' {
            continue
        } else {
            builder.WriteByte(char)
        }
    }

    if attribute.Name == "" {
        attribute.Name = builder.String()
    }
    return attribute
}

// buat mapping combinator ke nama relasi
func mapRelation(t string) string {
    switch t {
    case " ": return "descendant"
    case ">": return "child"
    case "+": return "adjacent_sibling"
    case "~": return "general_sibling"
    }
    return ""
}

```

4.1.1.7. matcher.go

Nama	Tipe	Deskripsi
MakeMatchFunc	Fungsi	Menerima string selector CSS, melakukan parsing sekali, dan mengembalikan fungsi bertipe MatchFunc yang digunakan untuk mencocokkan suatu node dengan selector tersebut (closure).
MatchSelector	Fungsi	Memanggil matchSelectorRecursive untuk memeriksa apakah suatu node cocok dengan selector yang sudah di-parse (dalam bentuk SelectorNode).
matchSelectorRecursive	Fungsi (rekursif)	Logika inti pencocokan selector dengan mempertimbangkan combinator (child, descendant, sibling). Menelusuri node dan selector node secara rekursif sesuai relasi.
matchNode	Fungsi	Memeriksa apakah suatu node cocok dengan satu komponen selector (tanpa combinator), meliputi tag, id, class, dan attribute selector.
matchAttributes	Fungsi	Memeriksa apakah attribute selector (misal [attr=value]) cocok dengan attribute node DOM. Mendukung operator =, ~, ~=,

```
package main

import "strings"

func MakeMatchFunc(selector string) MatchFunc {
    parsed := ParseSelector(selector)
    return func(node *Node, _ string) bool {
        return MatchSelector(node, parsed)
    }
}

func MatchSelector(dom *Node, selector *SelectorNode) bool {
```

```

        return matchSelectorRecursive(dom, selector)
    }

func matchSelectorRecursive(node *Node, sel *SelectorNode) bool {
    if node == nil {
        return false
    }

    // cek node sekarang apakah cocok
    if !matchNode(node, sel) {
        return false
    }

    // kalo paling kiri udah berarti selesai
    if sel.Prev == nil {
        return true
    }

    // lanjut sesuai relation
    switch sel.Relation {
    case "child":
        return matchSelectorRecursive(node.Parent, sel.Prev)

    case "descendant":
        cur := node.Parent
        for cur != nil {
            if matchSelectorRecursive(cur, sel.Prev) {
                return true
            }
            cur = cur.Parent
        }
        return false

    case "adjacent_sibling":
        return
        matchSelectorRecursive(node.ImmediateSiblingsBeforeNow(), sel.Prev)

    case "general_sibling":
        siblings := node.SiblingsBeforeNow()
        for i := len(siblings) - 1; i >= 0; i-- {
            if matchSelectorRecursive(siblings[i], sel.Prev) {
                return true
            }
        }
        return false
    }

    return false
}

func matchNode(dom *Node, selector *SelectorNode) bool {

```

```

    if dom == nil {
        return false
    }

    // tag
    if selector.Tag != "" && selector.Tag != "*" && selector.Tag
!= dom.Tag {
        return false
    }

    // id
    if selector.ID != "" && selector.ID != dom.ID {
        return false
    }

    // class
    for _, c := range selector.Classes {
        if !dom.HasClasses(c) {
            return false
        }
    }

    // attribute
    if !matchAttributes(dom, selector) {
        return false
    }

    return true
}

func matchAttributes(dom *Node, selector *SelectorNode) bool {
    for _, attr := range selector.Attributes {
        var val string
        var found bool

        // ambil dari dom
        if attr.Name == "id" {
            val = dom.ID
            found = dom.ID != ""
        } else if attr.Name == "class" {
            val = strings.Join(dom.Classes, " ")
            found = len(dom.Classes) > 0
        } else {
            val, found = dom.GetAttribute(attr.Name)
        }

        if !found {
            return false
        }

        a := val
    }

```

```

    b := attr.Value

// cek case insensitive
if attr.CaseInsensitive {
    a = strings.ToLower(a)
    b = strings.ToLower(b)
}

// cek logic operator
switch attr.Operator {
case "":
    continue

case "=":
    if a != b {
        return false
    }

case "~=":
    words := strings.Fields(a)
    ok := false
    for _, w := range words {
        if w == b {
            ok = true
            break
        }
    }
    if !ok {
        return false
    }

case "|=":
    if a != b && !strings.HasPrefix(a, b+"-") {
        return false
    }

case "^=":
    if b == "" || !strings.HasPrefix(a, b) {
        return false
    }

case "$=":
    if b == "" || !strings.HasSuffix(a, b) {
        return false
    }

case "*=":
    if b == "" || !strings.Contains(a, b) {
        return false
    }
}

```

```

    }
    return true
}

```

4.1.1.8. traverser.go

Nama	Tipe	Deskripsi
TraversalRes	Struct	Menampung hasil dari BFS/DFS: MatchedNodes (node yang cocok), VisitedNodes (semua node yang dikunjungi), VisCount (jumlah node dikunjungi), ElapsedTime, TraversalLog (slice StepLog), MaxDepthVisited (kedalaman maksimum yang dicapai saat traversal), FullMaxDepth (kedalaman maksimum seluruh pohon).
StepLog	Struct	Merekam satu langkah traversal: step ke-berapa, tag, id, class, attributes, depth, apakah matched, algoritma (bfs/dfs), dan NodeIndex (indeks unik node).
MatchFunc	Tipe fungsi	Signatur fungsi untuk mencocokkan node dengan selector. Digunakan oleh BFS/DFS.
maxFullTreeDepth	Fungsi	Menghitung kedalaman maksimum seluruh pohon dengan BFS.
BFS	Fungsi	Melakukan traversal Breadth First Search (queue). Menerima root, selector, fungsi match, topN, depth maksimal pohon, dan indexMap. Mengembalikan TraversalRes.
DFS	Fungsi	Melakukan traversal Depth First Search (stack). Menerima parameter sama seperti BFS. Mengembalikan TraversalRes.

AlgorithmType	Tipe string	Enum untuk algoritma: AlgoBFS dan AlgoDFS.
Search	Fungsi	Entry point utama. Menghitung maxFullTreeDepth, lalu memanggil BFS atau DFS sesuai algoType.

```

package main

import "time"

//nyimpan hasil dari proses traversal
type TraversalRes struct{
    MatchedNodes []*Node //nodes yang ngematch
    VisitedNodes []*Node //nodes visited
    VisCount int //total visited
    ElapsedTime time.Duration
    TraversalLog []StepLog //log detail setiap step traversal
    MaxDepthVisited int //depth terdalam selama transversal saat
ini
    FullMaxDepth int //depth maksimum dari seluruh tree
}

//Ngelog sebuah step dalam proses transversal
type StepLog struct{
    Step int
    N_Tag string
    N_ID string
    N_Classes []string
    N_Attributes map[string]string
    Depth int
    Is_matched bool
    Algorithm string //bfs atau dfs
    NodeIndex int
}

type MatchFunc func(node *Node, selector string) bool

//nyari maximum depth dari seluruh DOM tree
func maxFullTreeDepth(root *Node) int{
    if root == nil{
        return 0
    }
    maxDepth := 0
    queue := []*Node{root}
    for len(queue) > 0{
        node := queue[0]
        queue = queue[1:]
    }
}

```

```

        if d := node.Depth(); d > maxDepth{
            maxDepth = d
        }
        queue = append(queue, node.Children...)
    }
    return maxDepth
}

//dengan menggunakan referensi dari mediumnya Timothy Britt dan
//StackOverflow, algoritma ini menggunakan sebuah queue dengan konsep
//FIFO
func BFS(root *Node, selector string, matchF MatchFunc, topN int,
treeMaxDepth int, indexMap map[*Node]int) TraversalRes{
    start := time.Now()
    result := TraversalRes{
        MatchedNodes: []*Node{},
        VisitedNodes: []*Node{},
        TraversalLog: []StepLog{},
        FullMaxDepth: treeMaxDepth,
    }

    if root == nil{
        return result
    }

    queue := []*Node{root}
    visited := make(map[*Node]bool)
    step := 0
    maxDepthVisited := 0

    for len(queue) > 0{
        currentNode := queue[0]
        queue = queue[1:]
        if visited[currentNode]{
            continue
        }
        //mark as visited
        visited[currentNode] = true
        result.VisitedNodes = append(result.VisitedNodes,
currentNode)
        step++
        //track visited max depth
        depth := currentNode.Depth()
        if depth > maxDepthVisited{
            maxDepthVisited = depth
        }
        //cek apakah nodenya ngematch sama selector
        matched := matchF(currentNode, selector)
        //ambil nodeIndex dari map
        nodeIdx := indexMap[currentNode]
        //catat step kedalam transversal log

```

```

        result.TraversalLog = append(result.TraversalLog,
StepLog{
            Step: step,
            N_Tag: currentNode.Tag,
            N_ID: currentNode.ID,
            N_Classes: currentNode.Classes,
            N_Attributes: currentNode.Attributes,
            Depth: depth,
            Is_matched: matched,
            Algorithm: "bfs",
            NodeIndex: nodeIdx,
        })
        if matched{
            result.MatchedNodes = append(result.MatchedNodes,
currentNode)

            //berhenti pas jumlah nodes terpenuhi
            if topN != -1 && len(result.MatchedNodes) >= topN{
                break
            }
        }
        //children di enqueue, untuk bfs tidak ada reverse
        for _, child := range currentNode.Children{
            if !visited[child]{
                queue = append(queue, child)
            }
        }
    }
    result.VisCount = step
    result.ElapsedTime = time.Since(start)
    result.MaxDepthVisited = maxDepthVisited
    return result
}

//dengan menggunakan referensi dari mediumnya Timothy Britt dan
//StackOverflow, algoritma ini menggunakan sebuah stack LIFO
func DFS(root *Node, selector string, matchF MatchFunc, topN int,
treeMaxDepth int, indexMap map[*Node]int) TraversalRes{
    start := time.Now()
    result := TraversalRes{
        MatchedNodes: []*Node{},
        VisitedNodes: []*Node{},
        TraversalLog: []StepLog{},
        FullMaxDepth: treeMaxDepth,
    }

    if root == nil{
        return result
    }

    stack := []*Node{root}
    visited := make(map[*Node]bool)

```

```

step := 0
maxDepthVisited := 0

for len(stack) > 0{
    n := len(stack) - 1
    currentNode := stack[n]
    stack = stack[:n]
    if visited[currentNode]{
        continue
    }
    //mark as visited
    visited[currentNode] = true
    result.VisitedNodes = append(result.VisitedNodes,
currentNode)
    step++
    //track visited max depth
    depth := currentNode.Depth()
    if depth > maxDepthVisited{
        maxDepthVisited = depth
    }
    //cek apakah nodenya ngematch sama selector
    matched := matchF(currentNode, selector)
    //ambil nodeIndex dari map
    nodeIdx := indexMap[currentNode]
    //catat stepnya ke traversal log
    result.TraversalLog = append(result.TraversalLog,
StepLog{
        Step: step,
        N_Tag: currentNode.Tag,
        N_ID: currentNode.ID,
        N_Classes: currentNode.Classes,
        N_Attributes: currentNode.Attributes,
        Depth: depth,
        Is_matched: matched,
        Algorithm: "dfs",
        NodeIndex: nodeIdx,
    })
    if matched{
        result.MatchedNodes = append(result.MatchedNodes,
currentNode)
        if topN != -1 && len(result.MatchedNodes) >= topN{
            break
        }
    }
    //push children secara reverse biar leftmost di proses
    terlebih dahulu
    for i := len(currentNode.Children) - 1; i >= 0; i--{
        if !visited[currentNode.Children[i]]{
            stack = append(stack, currentNode.Children[i])
        }
    }
}

```

```

    }
    result.VisCount = step
    result.ElapsedTime = time.Since(start)
    result.MaxDepthVisited = maxDepthVisited
    return result
}

type AlgorithmType string
const(
    AlgoBFS AlgorithmType = "bfs"
    AlgoDFS AlgorithmType = "dfs"
)

//main entry point yang dipanggil dari handler, ngekomputasi dulu
full max depth tree, baru mulai bfs/dfs
func Search(root *Node, selector string, algoType AlgorithmType,
matchF MatchFunc, topN int, indexMap map[*Node]int) TraversalRes{
    maxDepth := maxFullTreeDepth(root)
    switch algoType{
    case AlgoBFS:
        return BFS(root, selector, matchF, topN, maxDepth,
indexMap)
    case AlgoDFS:
        return DFS(root, selector, matchF, topN, maxDepth,
indexMap)
    default:
        return BFS(root, selector, matchF, topN, maxDepth,
indexMap) //defaultnya ke bfs
    }
}

```

4.1.1.9. logger.go

Nama	Tipe	Deskripsi
PrintTraversalLogResult	Fungsi	Menulis log traversal ke file teks dengan nama filename. Header berisi algoritma, selector, timestamp. Kemudian mencetak setiap step (StepLog) dengan format: tag, id, class, depth, matched. Diakhiri summary: total visited, matched, elapsed time, max depth visited, max depth seluruh tree.
isEmpty	Fungsi (helper)	Mengembalikan string "-" jika string input kosong, selain itu mengembalikan string aslinya.

		Digunakan untuk ID yang kosong.
--	--	---------------------------------

```
package main
import(
    "fmt"
    "os"
    "strings"
    "time"
)

func PrintTraversalLogResult(result TraversalRes, selector string,
filename string) error{
    f, err := os.Create(filename)
    if err != nil{
        return fmt.Errorf("Gagal bikin logfile: %w", err)
    }
    defer f.Close()

    algo := "unknown"
    if len(result.TraversalLog) > 0{
        algo = result.TraversalLog[0].Algorithm
    }

    //header
    fmt.Fprintln(f, "=====Traversal Log=====")
    fmt.Fprintf(f, "Algorithm: %s\n", algo)
    fmt.Fprintf(f, "Selector: %s\n", selector)
    fmt.Fprintf(f, "Timestamp: %s\n",
time.Now().Format("2001-09-11 10:00:67"))
    fmt.Fprintln(f, strings.Repeat("-", 50))
    //steps
    for _, entry := range result.TraversalLog{
        classes := strings.Join(entry.N_Classes, " ")
        if classes == ""{
            classes = "-"
        }
        matchedString := "no"
        if entry.Is_matched{
            matchedString = "yes"
        }
        fmt.Fprintf(f, "Step %03d | Tag: %-8s | ID: %-12s |
Classes: %-20s | Depth: %d | Matched: %s\n", entry.Step,
entry.N_Tag, ifEmpty(entry.N_ID), classes, entry.Depth,
matchedString)
    }
    //summarynya
    fmt.Fprintln(f, strings.Repeat("-", 50))
    fmt.Fprintln(f, "=====Summary=====")
}
```

```

        fmt.Fprintf(f, "Total visited: %d\n", result.VisCount)
        fmt.Fprintf(f, "Total matched: %d\n",
len(result.MatchedNodes))
        fmt.Fprintf(f, "Elapsed time: %s\n", result.ElapsedTime)
        fmt.Fprintf(f, "Max depth visited in process: %d\n",
result.MaxDepthVisited)
        fmt.Fprintf(f, "Max depth of the whole tree: %d\n",
result.FullMaxDepth)
        return nil;
    }

func isEmpty(s string) string{
    if s == ""{
        return "-"
    }
    return s
}

```

4.1.1.10. lca.go (bonus)

Nama	Tipe	Deskripsi
maxLog	Konstanta	Batas maksimal $\log_2(n)$ yang digunakan untuk alokasi slice ancestor (diberi nilai 64 sebagai safety cap).
LCATable	Struct	Struktur utama untuk menyimpan data hasil preprocessing binary lifting: tIn, tOut, depth, ancestor (map dari node ke slice ancestor ke- 2^k), nodeList (urutan DFS), timer, dan log (nilai $\log_2$ dari jumlah node).
computeLog	Fungsi	Menghitung $\text{ceil}(\log_2(n))$ secara manual (tanpa math library) dengan perulangan. Minimal menghasilkan 1.
dfs	Method (LCATable)	Melakukan DFS traversal untuk mengisi tIn, tOut, depth, dan ancestor[v][0]. Juga mencatat semua node ke nodeList.
buildLCATable	Fungsi	Membangun LCATable dari root pohon DOM. Memanggil DFS, menghitung log, kemudian mengisi ancestor untuk

		k=1..log-1 menggunakan rumus binary lifting. Mengembalikan pointer ke LCATable yang siap dipakai.
isAnc	Method (LCATable)	Memeriksa apakah u adalah ancestor dari v berdasarkan nilai tIn dan tOut.
IsAncestor	Method (LCATable)	Versi publik dari isAnc (diekspos untuk API handler).
lca	Method (LCATable)	Mencari Lowest Common Ancestor dari dua node u dan v dengan binary lifting. Mengembalikan node LCA atau nil jika tidak valid.
nodeDepth	Method (LCATable)	Mengembalikan kedalaman node (dari tabel depth).
nodeDistance	Method (LCATable)	Menghitung jarak (jumlah edges) antara dua node u dan v menggunakan depth dan LCA.
kthAnc	Method (LCATable)	Mengembalikan ancestor ke-k (1 = parent, 2 = grandparent, dst). Jika k melebihi kedalaman node, mengembalikan nil.

```

package main
//algoritma lca menggunakan binary lifting ini mengambil referensi
dari https://cp-algorithms.com/graph/lca\_binary\_lifting.html serta
mengambil inspirasi dari
https://www.youtube.com/watch?v=oib-XsjFa-M
//berikut adalah tahapannya preprocessing untuk binary lifting:
//1. dfs traversal dulu untuk mencatat waktu masuk (tin) dan waktu
keluar (tout) tiap node
//2. buat tabel ancestor dimana nantinya ancestor[v][k] adalah
ancestor ke 2^k dari v
//untuk tiap query lca(u, v):
//1. cek dulu apakah salah satu dari u atau v adalah ancestor dari
yang lain, kalau iya itu lcanya
//2. kalau bukan, naikin ke atas sebesar 2^k langkah, berhenti
tepat sebelum ancestor u dan v jadi saudara
//3. lca = ancestor[u][0] (direct parent u setelah loop)

const maxLog = 64 //ini sekedar buat safety cap aja

//LCATable ini untuk nampung semua info preprocessed untuk query

```

```

lca nanti
type LCATable struct{
    tIn map[*Node]int //waktu masuk tiap node
    tOut map[*Node]int //waktu keluar tiap node
    depth map[*Node]int //depth tiap node
    ancestor map[*Node][]*Node //ancestor[v][k] itu ancestor ke
    2^k dari v
    nodeList []*Node //nyimpan semua node dalam urutan dfs untuk
    preprocessing
    timer int //global dfs timestamp
    log int //the computed log2(n)
}

//buat ngitung ceil(log2(n)) secara gabole pake library math,
minimal 1
func computeLog(n int) int{
    log := 1
    for (1 << log) <= n{
        log++
        if log >= maxLog{
            break
        }
    }
    return log
}

func (t *LCATable) dfs(v *Node, parent *Node){
    t.timer++;
    t.tIn[v] = t.timer
    t.nodeList = append(t.nodeList, v)
    //inisialisasi slice ancestor[v][0] dengan parent langsungnya
    anc := make([]*Node, 1)
    if parent != nil{
        anc[0] = parent
        t.depth[v] = t.depth[parent] + 1
    }else{
        anc[0] = nil //root gada parent
        t.depth[v] = 0
    }
    t.ancestor[v] = anc

    for _, child := range v.Children{
        t.dfs(child, v)
    }

    t.timer++
    t.tOut[v] = t.timer
}

//ngepreprocess dom tree dari root dan mengembalikan LCATable,
harus dipanggil sekali setelah parseHTML

```

```

func buildLCATable(root *Node) *LCATable{
    table := &LCATable{
        tIn: make(map[*Node]int),
        tOut: make(map[*Node]int),
        depth: make(map[*Node]int),
        ancestor: make(map[*Node][]*Node),
        nodeList: []*Node{},
        timer: 0,
        log: 1, //akan diupdate setelah dfs
    }

    //dfs dulu buat isi tIn, tOut, depth, dan ancestor[v][0]
    table.dfs(root, nil)

    //lalu hitung log berdasarkan jumlah node
    n := len(table.nodeList)
    table.log = computeLog(n)

    //expand semua slice ancestor hingga panjang log full, terus
    isi ancestor[v][k] untuk k = 1...log-1
    //ancestor[v][k] = ancestor[ancestor[v][k-1]][k-1]
    for _, node := range table.nodeList{
        anc := table.ancestor[node]
        //kembangkan slicenya jadi panjang log (karena sebelumnya
        diinisialisasi panjangnya 1 dari dfs)
        for len(anc) < table.log{
            anc = append(anc, nil)
        }
        table.ancestor[node] = anc
    }
    for k := 1; k < table.log; k++){
        for _, node := range table.nodeList{
            anc := table.ancestor[node]
            if anc[k-1] != nil{
                parAnc := table.ancestor[anc[k-1]]
                if k - 1 < len(parAnc){
                    anc[k] = parAnc[k-1]
                }
            }
            table.ancestor[node] = anc
        }
    }

    return table
}

//ancestor validator
func (t *LCATable) isAnc(u, v *Node) bool{
    return t.tIn[u] <= t.tIn[v] && t.tOut[u] >= t.tOut[v]
}

```

```

//IsAncestor ini mengekspos fungsi isAnc secara publik untuk API
handler
func (t *LCATable) IsAncestor(u, v *Node) bool{
    return t.isAnc(u, v)
}

func (t *LCATable) lca(u, v *Node) *Node{
    if u == nil || v == nil{
        return nil
    }
    //cek dulu apakah kedua node exist dalam tabel
    if _, ok := t.tIn[u]; !ok{
        return nil
    }
    if _, ok := t.tIn[v]; !ok{
        return nil
    }

    //cek apakah salah satu node adalah ancestor dari yang lain
    if t.isAnc(u, v){
        return u
    }
    if t.isAnc(v, u){
        return v
    }

    //naikkan u ke atas sebesar 2^k langkah sampai tepat sebelum
    ancestor u dan v jadi saudara (cari node tertinggi yang bukan
    ancestor si v)
    for k := t.log - 1; k >= 0; k--{
        anc := t.ancestor[u]
        if k < len(anc) && anc[k] != nil && !t.isAnc(anc[k], v){
            u = anc[k]
        }
    }

    //lca adalah direct parent dari u setelah loop
    return t.ancestor[u][0]
}

func (t *LCATable) nodeDepth(n *Node) int{
    return t.depth[n]
}

//ngitung jumlah edges antara u dan v
func (t *LCATable) nodeDistance(u, v *Node) int{
    lca := t.lca(u, v)
    if lca == nil{
        return -1
    }
    return t.depth[u] + t.depth[v] - (2*t.depth[lca])
}

```

```

}

//ngembaliin k-th ancestor dari node n (1 untuk parent, 2 untuk
grandparent, etc). ngembaliin nil kalau k melebihi dari depth node
func (t *LCATable) kthAnc(n *Node, k int) *Node{
    if n == nil || k < 0{
        return nil
    }
    if t.depth[n] < k{
        return nil
    }
    for j := t.log - 1; j >= 0; j--{
        if k >= (1 << j){
            anc := t.ancestor[n]
            if j < len(anc) && anc[j] == nil{
                return nil
            }
            n = anc[j]
            k -= 1 << j
        }
    }
    return n
}

```

4.1.2. Interface Program

DOM Traversal Visualizer

Tugas Besar 2 IF2211 - Strategi Algoritma

Konfigurasi

CSS Selector

LCA

Input HTML

URL Website

https://example.com

Algoritma

Breadth First Search (BFS)

CSS Selector

Contoh: div.container > p

Top N Results (-1 = semua)

-1

☒ Tampilkan animasi traversal

☒ Simpan log traversal ke file

Search

Visualisasi DOM Tree

Hasil parsing tree akan tampil di sini...

DOM Traversal Visualizer

Tugas Besar 2 IF2211 - Strategi Algoritma

Konfigurasi

CSS Selector

LCA

Input HTML

URL Website

https://example.com

Build DOM Tree

Klik dua node di DOM tree untuk memilih Node A dan Node B

Node A: -

Node B: -

Reset Pilihan

Search LCA

Visualisasi DOM Tree

Hasil parsing tree akan tampil di sini...

Input HTML

URL Website

https://example.com

Algoritma

Breadth First Search (BFS)

CSS Selector

Contoh: div.container > p

Top N Results (-1 = semua)

-1

☒ Tampilkan animasi traversal

Search

Hasil parsing tree akan tampil di sini...

Visited Nodes

Max Depth

Time

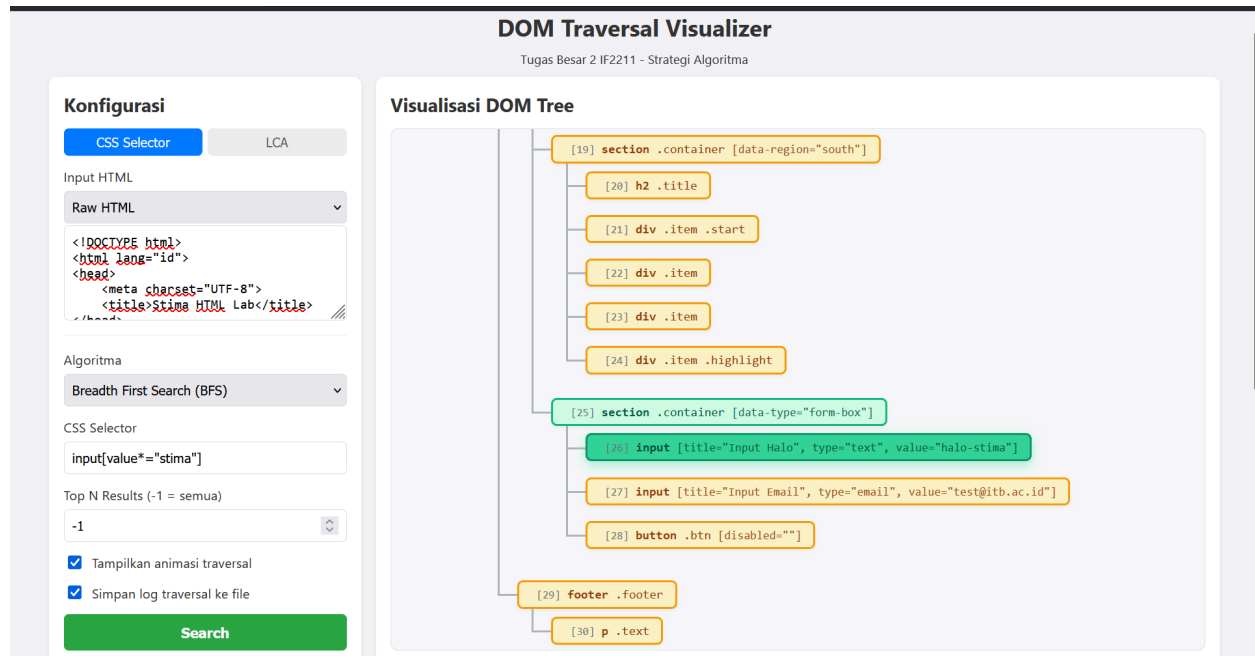
Matches

Log Traversal

Belum ada aktivitas penelusuran.

4.2. Pengujian

4.2.1. Testcase 1



Input raw html:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Stima HTML Lab</title>
</head>
<body>
  <header id="navbar" class="header-main">
    <nav>
      <ul class="nav-list">
        <li><a href="/" class="link active">Home</a></li>
        <li><a href="/docs" class="link">Docs</a></li>
      </ul>
    </nav>
  </header>

  <main id="app-content">
    <section class="container" data-region="north">
      <div class="card primary">
        <h2 class="title">Main Card</h2>
        <div class="inner-box">
          <p class="text">Target 1: Keturunan dalam</p>
        </div>
      </div>
    </section>
  </main>
</body>
</html>
```

```

        <p class="text">Target 2: Anak langsung section</p>
    </section>

    <section class="container" data-region="south">
        <h2 class="title">Sibling Test</h2>
        <div class="item start">Item 1 (Start)</div>
        <div class="item">Item 2 (Adjacent Sibling)</div>
        <div class="item">Item 3 (General Sibling)</div>
        <div class="item highlight">Item 4 (General Sibling +
Class)</div>
    </section>

    <section class="container" data-type="form-box">
        <input type="text" value="halo-stima" title="Input
Halo">
        <input type="email" value="test@itb.ac.id" title="Input
Email">
        <button class="btn" disabled>Submit</button>
    </section>
</main>

<footer class="footer">
    <p class="text">Copyright 2026</p>
</footer>
</body>
</html>

```

Output log:

```

=====Traversal Log=====
Algorithm: bfs
Selector: input[value*="stima"]
Timestamp: 25004-09-44 40:00:67
-----
Step 001 | Tag: html      | ID: -          | Classes: -
| Depth: 0 | Matched: no
Step 002 | Tag: head        | ID: -          | Classes: -
| Depth: 1 | Matched: no
Step 003 | Tag: body        | ID: -          | Classes: -
| Depth: 1 | Matched: no
Step 004 | Tag: meta        | ID: -          | Classes: -
| Depth: 2 | Matched: no
Step 005 | Tag: title       | ID: -          | Classes: -
| Depth: 2 | Matched: no
Step 006 | Tag: header      | ID: navbar     | Classes: header-main
| Depth: 2 | Matched: no
Step 007 | Tag: main        | ID: app-content | Classes: -
| Depth: 2 | Matched: no
Step 008 | Tag: footer      | ID: -          | Classes: footer
| Depth: 2 | Matched: no

```

Step 009	Tag: nav	ID: -	Classes: -
Depth: 3	Matched: no		
Step 010	Tag: section	ID: -	Classes: container
Depth: 3	Matched: no		
Step 011	Tag: section	ID: -	Classes: container
Depth: 3	Matched: no		
Step 012	Tag: section	ID: -	Classes: container
Depth: 3	Matched: no		
Step 013	Tag: p	ID: -	Classes: text
Depth: 3	Matched: no		
Step 014	Tag: ul	ID: -	Classes: nav-list
Depth: 4	Matched: no		
Step 015	Tag: div	ID: -	Classes: card primary
Depth: 4	Matched: no		
Step 016	Tag: p	ID: -	Classes: text
Depth: 4	Matched: no		
Step 017	Tag: h2	ID: -	Classes: title
Depth: 4	Matched: no		
Step 018	Tag: div	ID: -	Classes: item start
Depth: 4	Matched: no		
Step 019	Tag: div	ID: -	Classes: item
Depth: 4	Matched: no		
Step 020	Tag: div	ID: -	Classes: item
Depth: 4	Matched: no		
Step 021	Tag: div	ID: -	Classes: item
highlight	Depth: 4	Matched: no	
Step 022	Tag: input	ID: -	Classes: -
Depth: 4	Matched: yes		
Step 023	Tag: input	ID: -	Classes: -
Depth: 4	Matched: no		
Step 024	Tag: button	ID: -	Classes: btn
Depth: 4	Matched: no		
Step 025	Tag: li	ID: -	Classes: -
Depth: 5	Matched: no		
Step 026	Tag: li	ID: -	Classes: -
Depth: 5	Matched: no		
Step 027	Tag: h2	ID: -	Classes: title
Depth: 5	Matched: no		
Step 028	Tag: div	ID: -	Classes: inner-box
Depth: 5	Matched: no		
Step 029	Tag: a	ID: -	Classes: link active
Depth: 6	Matched: no		
Step 030	Tag: a	ID: -	Classes: link
Depth: 6	Matched: no		
Step 031	Tag: p	ID: -	Classes: text
Depth: 6	Matched: no		

=====Summary=====

Total visited: 31

Total matched: 1

Elapsed time: 0s

Max depth visited in process: 6
Max depth of the whole tree: 6

4.2.2. Testcase 2

DOM Traversal Visualizer
Tugas Besar 2 IF2211 - Strategi Algoritma

Konfigurasi

CSS Selector **LCA**

Input HTML

Raw HTML

```
<footer class="main-footer">
  <p>&copy; 2026 STEI-K
ITB</p>
</footer>
</body>
</html>
```

Algoritma

Breadth First Search (BFS)

CSS Selector

.card

Top N Results (-1 = semua)

-1

☒ Tampilkan animasi traversal

☒ Simpan log traversal ke file

Search

Visualisasi DOM Tree

Input raw html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sistem Informasi Akademik</title>
</head>
<body>
  <header id="top-header">
    <nav class="navbar">
      <div class="container">
        <a href="/" class="brand">SIMAK-ITB</a>
        <ul class="nav-links">
          <li><a href="/dashboard">Dashboard</a></li>
          <li><a href="/profile"
class="active">Profil</a></li>
          <li><a href="/courses">Mata Kuliah</a></li>
        </ul>
      </div>
    </nav>
  </header>
```

```

<main id="main-content">
  <section class="user-info">
    <div class="card" data-type="personal">
      <h2 class="title">Data Mahasiswa</h2>
      <div class="details">
        <p>Nama: <span>Ganesha</span></p>
        <p>NIM: <span>13524000</span></p>
        <p class="status highlight"
data-verified="true">Status: Aktif</p>
      </div>
    </div>
  </section>

  <section class="academic-info">
    <div class="card" data-type="academic">
      <h2 class="title">Informasi Akademik</h2>
      <table>
        <tr>
          <th>Kode</th>
          <th>Mata Kuliah</th>
          <th>Nilai</th>
        </tr>
        <tr>
          <td>IF2211</td>
          <td>Strategi Algoritma</td>
          <td>A</td>
        </tr>
        <tr>
          <td>IF2120</td>
          <td>Matematika Diskrit</td>
          <td>AB</td>
        </tr>
      </table>
    </div>
  </section>

  <aside class="sidebar">
    <div class="card" data-type="notification">
      <h2 class="title">Notifikasi</h2>
      <ul>
        <li class="urgent">Tugas Stima Deadline
Besok!</li>
        <li>Kuliah umum jam 10:00</li>
        <li class="highlight">Pembayaran UKT
Dibuka</li>
      </ul>
    </div>
  </aside>
</main>

<footer class="main-footer">

```

```
<p>&copy; 2026 STEI-K ITB</p>
</footer>
</body>
</html>
```

Output log :

```
=====Traversal Log=====
Algorithm: bfs
Selector: .card
Timestamp: 25004-09-44 40:00:67
-----
Step 001 | Tag: html      | ID: -          | Classes: -
| Depth: 0 | Matched: no
Step 002 | Tag: head       | ID: -          | Classes: -
| Depth: 1 | Matched: no
Step 003 | Tag: body       | ID: -          | Classes: -
| Depth: 1 | Matched: no
Step 004 | Tag: meta       | ID: -          | Classes: -
| Depth: 2 | Matched: no
Step 005 | Tag: title      | ID: -          | Classes: -
| Depth: 2 | Matched: no
Step 006 | Tag: header     | ID: top-header | Classes: -
| Depth: 2 | Matched: no
Step 007 | Tag: main       | ID: main-content | Classes: -
| Depth: 2 | Matched: no
Step 008 | Tag: footer     | ID: -          | Classes: main-footer
| Depth: 2 | Matched: no
Step 009 | Tag: nav        | ID: -          | Classes: navbar
| Depth: 3 | Matched: no
Step 010 | Tag: section    | ID: -          | Classes: user-info
| Depth: 3 | Matched: no
Step 011 | Tag: section    | ID: -          | Classes:
academic-info      | Depth: 3 | Matched: no
Step 012 | Tag: aside      | ID: -          | Classes: sidebar
| Depth: 3 | Matched: no
Step 013 | Tag: p          | ID: -          | Classes: -
| Depth: 3 | Matched: no
Step 014 | Tag: div        | ID: -          | Classes: container
| Depth: 4 | Matched: no
Step 015 | Tag: div        | ID: -          | Classes: card
| Depth: 4 | Matched: yes
Step 016 | Tag: div        | ID: -          | Classes: card
| Depth: 4 | Matched: yes
Step 017 | Tag: div        | ID: -          | Classes: card
| Depth: 4 | Matched: yes
Step 018 | Tag: a          | ID: -          | Classes: brand
| Depth: 5 | Matched: no
Step 019 | Tag: ul         | ID: -          | Classes: nav-links
```

Depth: 5	Matched: no		
Step 020	Tag: h2	ID: -	Classes: title
Depth: 5	Matched: no		
Step 021	Tag: div	ID: -	Classes: details
Depth: 5	Matched: no		
Step 022	Tag: h2	ID: -	Classes: title
Depth: 5	Matched: no		
Step 023	Tag: table	ID: -	Classes: -
Depth: 5	Matched: no		
Step 024	Tag: h2	ID: -	Classes: title
Depth: 5	Matched: no		
Step 025	Tag: ul	ID: -	Classes: -
Depth: 5	Matched: no		
Step 026	Tag: li	ID: -	Classes: -
Depth: 6	Matched: no		
Step 027	Tag: li	ID: -	Classes: -
Depth: 6	Matched: no		
Step 028	Tag: li	ID: -	Classes: -
Depth: 6	Matched: no		
Step 029	Tag: p	ID: -	Classes: -
Depth: 6	Matched: no		
Step 030	Tag: p	ID: -	Classes: -
Depth: 6	Matched: no		
Step 031	Tag: p	ID: -	Classes: status
highlight	Depth: 6	Matched: no	
Step 032	Tag: tr	ID: -	Classes: -
Depth: 6	Matched: no		
Step 033	Tag: tr	ID: -	Classes: -
Depth: 6	Matched: no		
Step 034	Tag: tr	ID: -	Classes: -
Depth: 6	Matched: no		
Step 035	Tag: li	ID: -	Classes: urgent
Depth: 6	Matched: no		
Step 036	Tag: li	ID: -	Classes: -
Depth: 6	Matched: no		
Step 037	Tag: li	ID: -	Classes: highlight
Depth: 6	Matched: no		
Step 038	Tag: a	ID: -	Classes: -
Depth: 7	Matched: no		
Step 039	Tag: a	ID: -	Classes: active
Depth: 7	Matched: no		
Step 040	Tag: a	ID: -	Classes: -
Depth: 7	Matched: no		
Step 041	Tag: span	ID: -	Classes: -
Depth: 7	Matched: no		
Step 042	Tag: span	ID: -	Classes: -
Depth: 7	Matched: no		
Step 043	Tag: th	ID: -	Classes: -
Depth: 7	Matched: no		
Step 044	Tag: th	ID: -	Classes: -
Depth: 7	Matched: no		

```

Step 045 | Tag: th      | ID: -      | Classes: -
| Depth: 7 | Matched: no
Step 046 | Tag: td      | ID: -      | Classes: -
| Depth: 7 | Matched: no
Step 047 | Tag: td      | ID: -      | Classes: -
| Depth: 7 | Matched: no
Step 048 | Tag: td      | ID: -      | Classes: -
| Depth: 7 | Matched: no
Step 049 | Tag: td      | ID: -      | Classes: -
| Depth: 7 | Matched: no
Step 050 | Tag: td      | ID: -      | Classes: -
| Depth: 7 | Matched: no
Step 051 | Tag: td      | ID: -      | Classes: -
| Depth: 7 | Matched: no
-----
=====Summary=====
Total visited: 51
Total matched: 3
Elapsed time: 0s
Max depth visited in process: 7
Max depth of the whole tree: 7

```

4.2.3. Testcase 3

Konfigurasi

CSS Selector

LCA

Input HTML

URL Website

https://en.cppreference.com/w/cpp/thread/th

Algoritma

Breadth First Search (BFS)

CSS Selector

div#mw-navigation div#mw-panel div.portal[

Top N Results (-1 = semua)

100

☐ Tampilkan animasi traversal
☒ Simpan log traversal ke file

Search

Visualisasi DOM Tree

[0] html {client-nojs {dir="ltr", lang="en"}}

[1] head

[2] meta {charset="UTF-8"}

[3] title

[4] script

[5] script

[6] link {href="/load.php?lang=en&modules=ext.pygments%7Cskins.cppreference2.styles&only=styles&skin=cppreference2&raw=1&skin=cppreference2"}

[7] script {async="", src="/load.php?lang=en&modules=startup&only=scripts&raw=1&skin=cppreference2"}

[8] meta {content="", name="ResourceLoaderDynamicStyles"}

[9] link {href="/load.php?lang=en&modules=site.styles&only=styles&skin=cppreference2", rel="stylesheet"}

Visited Nodes

1611

Max Depth

28

Time

3.031 ms

Matches

100

Input url website: <https://en.cppreference.com/w/cpp/thread/thread.html>

Output log:

```

=====Traversal Log=====

```

```

Algorithm: bfs
Selector: div#mw-navigation div#mw-panel
div.portal[role="navigation"] ul li a[href*="thread"]
Timestamp: 25004-09-44 40:00:67
-----
Step 001 | Tag: html      | ID: -          | Classes: client-nojs
| Depth: 0 | Matched: no
Step 002 | Tag: head      | ID: -          | Classes: -
| Depth: 1 | Matched: no
Step 003 | Tag: body      | ID: -          | Classes: mediawiki
ltr sitedir-ltr mw-hide-empty-elt ns-0 ns-subject
page-cpp_thread_thread rootpage-cpp skin-cppreference2 action-view
cpp-navbar | Depth: 1 | Matched: no
Step 004 | Tag: meta      | ID: -          | Classes: -
| Depth: 2 | Matched: no
Step 005 | Tag: title     | ID: -          | Classes: -
| Depth: 2 | Matched: no
.
.
.
Step 1607 | Tag: a         | ID: -          | Classes: new
| Depth: 28 | Matched: yes
Step 1608 | Tag: span      | ID: -          | Classes: t-lines
| Depth: 28 | Matched: no
Step 1609 | Tag: a         | ID: -          | Classes: -
| Depth: 28 | Matched: yes
Step 1610 | Tag: span      | ID: -          | Classes: t-lines
| Depth: 28 | Matched: no
Step 1611 | Tag: a         | ID: -          | Classes: -
| Depth: 28 | Matched: yes
-----
=====Summary=====
Total visited: 1611
Total matched: 100
Elapsed time: 3.031ms
Max depth visited in process: 28
Max depth of the whole tree: 31

```

4.2.4. Testcase 4

Konfigurasi

CSS Selector

LCA

Input HTML

URL Website

https://en.cppreference.com/w/cpp/thread/th

Algoritma

Depth First Search (DFS)

CSS Selector

div#mw-navigation div#mw-panel div.portal[r

Top N Results (-1 = semua)

100

☐ Tampilkan animasi traversal

☒ Simpan log traversal ke file

Search

Visualisasi DOM Tree

[0] html .client-nojs [dir="ltr", lang="en"]

[1] head

[2] meta [charset="UTF-8"]

[3] title

[4] script

[5] script

[6] link [href="/load.php?lang=en&modules=ext.pygments%7Cskins.cppreference2.styles&only=styles&skin=cpp

[7] script [async="", src="/load.php?lang=en&modules=startup&only=scripts&raw=1&skin=cppreference2"]

[8] meta [content="", name="ResourceLoaderDynamicStyles"]

[9] link [href="/load.php?lang=en&modules=site.styles&only=styles&skin=cppreference2", rel="stylesheet"]

Visited Nodes

1529

Max Depth

31

Time

1.339 ms

Matches

100

Input url website: <https://en.cppreference.com/w/cpp/thread/thread.html>

Output log:

```
=====Traversal Log=====
Algorithm: dfs
Selector: div#mw-navigation div#mw-panel
div.portal[role="navigation"] ul li a[href*="thread"]
Timestamp: 25004-09-44 40:00:67
-----
Step 001 | Tag: html      | ID: -          | Classes: client-nojs
| Depth: 0 | Matched: no
Step 002 | Tag: head           | ID: -          | Classes: -
| Depth: 1 | Matched: no
Step 003 | Tag: meta           | ID: -          | Classes: -
| Depth: 2 | Matched: no
Step 004 | Tag: title          | ID: -          | Classes: -
| Depth: 2 | Matched: no
Step 005 | Tag: script         | ID: -          | Classes: -
| Depth: 2 | Matched: no
.
.
.
Step 1525 | Tag: span           | ID: -          | Classes: -
| Depth: 19 | Matched: no
Step 1526 | Tag: span           | ID: -          | Classes: t-mark-rev
t-since-cxx11 | Depth: 20 | Matched: no
Step 1527 | Tag: div            | ID: -          | Classes: -
| Depth: 11 | Matched: no
```

```

Step 1528 | Tag: span      | ID: -          | Classes: editsection
noprint plainlinks | Depth: 12 | Matched: no
Step 1529 | Tag: a      | ID: -          | Classes: external
text      | Depth: 13 | Matched: yes
-----
=====Summary=====
Total visited: 1529
Total matched: 100
Elapsed time: 1.3386ms
Max depth visited in process: 31
Max depth of the whole tree: 31

```

4.2.5. Testcase 5

DOM Traversal Visualizer
 Tugas Besar 2 IF2211 - Strategi Algoritma

Konfigurasi

CSS Selector

LCA

Input HTML
 URL Website ▼
 https://www.geeksforgeeks.org/dsa/differenc

Build DOM Tree

Klik dua node di DOM tree untuk memilih Node A dan Node B
 ● Node A: Node 156
 ● Node B: Node 157

Reset Pilihan

Search LCA

LCA: <div>
 .ArticlePagePostLayout_containerFluid_articlePage
 (Depth 8, Index 151)

Visualisasi DOM Tree

```

[149] div
[150] div #ArticlePagePostLayout_home-page__1dC9q .ArticlePagePostLayout_containerFluid_q38gg
[151] div .ArticlePagePostLayout_containerFluid_articlePageFlex_usibP
[152] div #LeftBarOutsideIndia_sidebar_OI__n_7R4 .sidebar-wrapper
[153] div #GFG_AD_Leftsidebar_300x250_chained_1 [style="margin-bottom:10px"]
[154] div #GFG_AD_Leftsidebar_300x250_chained_2 [style="margin-bottom:10px"]
[155] div #GFG_AD_Leftsidebar_300x250_chained_3 [style="margin-bottom:10px"]
[156] div #GFG_AD_Leftsidebar_300x600_chained_1 [style="margin-bottom:10px"]
[157] div .ArticlePagePostLayout_containerFluid_articlePageFlex_article_viewer__83Rkj .article--viewer .Ar
[158] div [style="display:flex;flex-direction:column"]
[159] div
[160] div

```

BAB V Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil implementasi yang telah dilakukan, aplikasi DOM Traversal Visualizer berhasil dikembangkan dengan memanfaatkan struktur data pohon yang direpresentasikan melalui tipe Node pada backend. Struktur ini menyimpan informasi penting seperti tag, id, class, atribut, serta relasi parent-child yang digunakan sebagai dasar pembentukan DOM Tree melalui fungsi `parseHTML()` pada modul `htmlparser.go`. Proses parsing HTML dilakukan menggunakan tokenizer dari library `golang.org/x/net/html`, yang kemudian disusun menjadi struktur tree menggunakan pendekatan stack. Validasi struktur dilakukan melalui fungsi `ValidateHTML()` untuk memastikan konsistensi hubungan antar node sebelum dilakukan proses traversal.

Dalam pencarian elemen berdasarkan CSS selector, sistem menggunakan fungsi `Search()` sebagai entry point yang akan memanggil algoritma BFS atau DFS sesuai pilihan pengguna. Implementasi BFS menggunakan struktur data queue dengan prinsip FIFO, sedangkan DFS menggunakan stack dengan prinsip LIFO, sebagaimana diimplementasikan pada fungsi `BFS()` dan `DFS()` di modul `traverser.go`. Selama proses traversal, setiap langkah dicatat dalam struktur `StepLog`, yang kemudian dikonversi menjadi `TraversalLogEntry` untuk ditampilkan ke frontend. Mekanisme pencocokan selector dilakukan melalui fungsi `MakeMatchFunc()` dan `matchSelectorRecursive()` pada modul `matcher.go`, yang mendukung berbagai jenis selector seperti tag, id, class, attribute, serta combinator. Pendekatan ini memungkinkan sistem untuk melakukan pencarian elemen secara fleksibel sesuai dengan standar CSS selector.

Selain itu, fitur pencarian Lowest Common Ancestor (LCA) diimplementasikan menggunakan teknik Binary Lifting melalui struktur `LCATable`. Proses preprocessing dilakukan menggunakan DFS untuk mencatat waktu masuk (`tin`), waktu keluar (`tout`), serta membangun tabel ancestor. Hal ini memungkinkan pencarian LCA dilakukan secara efisien dengan kompleksitas waktu $O(\log N)$, yang diimplementasikan pada fungsi `lca()` dalam modul `lca.go`. Hasil pengujian menunjukkan bahwa sistem mampu membangun DOM Tree dengan benar, melakukan traversal menggunakan BFS dan DFS secara akurat, serta menghasilkan log penelusuran yang detail sesuai dengan setiap langkah algoritma. Fitur LCA juga mampu menentukan ancestor terdekat dari dua node dengan tepat.

Secara keseluruhan, integrasi antara proses parsing HTML, struktur data pohon, algoritma traversal, serta teknik Binary Lifting menghasilkan sistem yang tidak hanya memenuhi spesifikasi tugas, tetapi juga efisien dan modular dalam implementasinya.

5.2. Saran

- ★ Mirza: saya harap tubes 3 tenggat nya tidak mendekati ataupun saat UAS dan tidak bertabrakan dengan tenggat tugas mata kuliah lain.
- ★ Fira: banyak kali tubes ini Ya Allah
- ★ Vari: jujur udah burnout parah _('T')_ pls tubes 3 nya dibikin seru dan menarik biar semangat dikit (tapi tolong ttp manusiawi yh-) 31 || ∞ ∪ || ∞)

LAMPIRAN

Checklist Spesifikasi Program

No.	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi		✓
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading		✓
13	[Bonus] Implementasi LCA Binary Lifting	✓	

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Varistha Devi



Mirza Tsabita Wafa' Ana

A handwritten signature in black ink, appearing to read 'Safira', with a stylized flourish at the end.

Safira Berlianti

Pranala

Link Repo Github: https://github.com/Beeziebloop/Tubes2_when-yh-libur

Link video: [video youtube](#)

DAFTAR PUSTAKA

- https://www.w3schools.com/cssref/css_selectors.php
- <https://medium.com/swlh/depth-first-and-breadth-first-dom-traversal-explained-for-real-86244fbf9854>
- <https://stackoverflow.com/questions/39503861/javascript-only-dom-tree-traversal-dfs-and-bfs>
- https://cp-algorithms.com/graph/lca_binary_lifting.html
- <https://www.youtube.com/watch?v=oib-XsjFa-M>